I want an animated explainer made in the "Engineering Blueprint" visual style (Hand-Drawn). Below is the complete source code of a reference animation in this exact style, plus a small sound-effects kit. Use them as your style guide and rebuild it around my content.

MY CONTENT:
[Describe your topic, or paste your script here. If this is left as is, ask me for it before building.]

STORYBOARD FIRST: before building the animation, make a VISUAL storyboard and wait for my OK.
- One HTML page (show it as an artifact if you can) with a grid of panels, one per beat (about 6-10). Each panel is a canvas showing that beat's key frame drawn in this style: use the H library and the reference's own drawing, texture and finishing functions so the panels already look like the finished piece. Still frames only, no animation or sound yet.
- Under each panel: the beat number, its start and end time, the exact on-screen text, how it gets there from the previous panel (continuous: which elements carry over and how they move or change; or a scene change: which transition, and why the story wants one), and its key sounds.
- Below the grid, list any facts or numbers you had to assume.
When I approve it (or after my changes), build the full animation exactly as storyboarded, reusing the storyboard's drawing code.

WHAT TO BUILD
One self-contained HTML file (show it as an artifact if you can):
- A 1280x720 canvas, scaled to fit the window with letterboxing, and a small control bar: play/pause, a scrub slider, the current time, a sound on/off toggle, and loop on by default.
- A new `explainer(ctx, W, H_, t)` that tells MY CONTENT in this style. Aim for 12-30 seconds unless my content needs more; set the length in a DURATION constant.
- A `scoreSound(k)` function that schedules the sound effects with the SOUND kit below (see SOUND). Play it in sync from the player: on play call `stopSound = SOUND.live(scoreSound, currentT, { voiceover: false })`, and call `stopSound()` on pause, on seek and when the sound toggle is turned off.
- Include the H helper library and the SOUND kit below unchanged, and keep the reference's own helper functions where they are useful.
- Load these Google Fonts before the first draw (await document.fonts.load for each spec you use): IBM Plex Mono, Space Mono
  <link rel="stylesheet" href="https://fonts.googleapis.com/css2?family=IBM+Plex+Mono:wght@400;600&family=Space+Mono:wght@400;700&display=block">
- Expose `window.seek = (t) => { /* draw frame at time t */ }`, `window.DURATION` and `window.scoreSound`, so the animation can be rendered frame by frame to video later.

HOW TO STAY IN THE STYLE
- Reuse the reference's rendering techniques, palette, textures, lettering, line quality, arrows, labels, number treatment, transitions and timing feel. Change the story, the text and the things being drawn; do not change the medium.

- Draw every subject accurately for my topic. Don't carry over the reference's props and drawing recipes (its characters, robots, cats and dogs, rings, icons) unless they genuinely fit, and double-check that the science and facts shown in the visuals are right.
- The surface texture and finishing passes are what make the style read as the real medium (paper grain, ink pooling, misregistration, grain, glow, scanlines and so on). Copy those functions from the reference and run them on every frame at the same strength. Don't simplify or skip them to save effort.
- Every frame is a pure function of t: create all randomness from H.rng(seed) / H.makeNoise(seed) inside the draw (never Math.random), so the same t always gives the same frame. Caching static textures across frames is fine.
- It is video, often watched on a phone: at 1280x720 keep headlines >= 44px, every label or caption the story depends on >= 28px (including small annotations and the corner header), key numbers >= 64px, and frame each beat so it fills most of the screen. Keep frames fast (well under 100 ms each if you can).
- In the reference, `explainer()` is the explainer (it explains "How does AI learn?"), `draw()` is the single hero frame of the style, and `explainerSound(k)` is its sound design. Study them, but do not reuse that topic unless my content asks for it.

STRUCTURE
- Structure it like a real explainer, not a title card: a hook that gets out of the way within ~2 s, then beats where something changes every 1-2 s; arrows that draw on to show cause and flow; labels that land on what they label; any full scene change done the medium's own way (see the reference); an emphasis move on the key moment; a short takeaway held ~1 s at the end.

FLOW BETWEEN BEATS
- Decide beat by beat what the story needs. A new beat does not automatically mean a new scene.
- When the next idea builds on the last, keep it continuous: elements that matter stay on screen and move, grow, re-arrange or transform into the next idea (a label becomes the next heading, a counter grows into a chart, the camera travels to a new part of the same scene, a shape morphs into the next one).
- When the story really moves on (a new topic, a jump in time or place, a before/after contrast, a reveal), a full scene change is the right call. Do it the medium's own way, as the reference does.
- Aim for a mix that fits the content, not a rule: avoid both a hard cut on every beat and forcing everything into one unbroken shot. The reference's own transitions show how this medium changes scene; its scene count is not a template to copy.

SOUND (no voiceover: a fuller mix)
- Sync is everything: a sound starts on the exact frame its event appears, using the same times as the picture.
- Score the events that land: things popping or stamping on, labels landing, arrows completing, scene transitions, a wrong/right moment, the key number (the biggest hit) and the takeaway landing (a resolving sound).
- Drawing makes its medium's noise while it draws (marker, chalk, pen strokes matched to the draw-on length; typing while text types; chip blips while game dialog prints). Counters tick at most ~12 times a second, slowing as they settle.

- A quiet ambience bed suited to the medium is welcome, and music only if it suits the style (k.music, gain 0.6 or less).
- Keep a hierarchy: effects in front, any bed or music clearly underneath, no more than ~3 sounds at once, and small silences so the big hits read. Medium-true palette, not generic swooshes.

If you are Claude Code with a shell: render an MP4 by loading the file in headless Chrome (e.g. puppeteer), calling window.seek(i/30) for each frame, capturing the canvas and piping the frames into ffmpeg. Render the effects to a WAV with `SOUND.renderOffline(window.scoreSound, DURATION, 48000, { voiceover: false })` (it returns interleaved stereo floats) and mux it in.

STYLE CARD
Name: Engineering Blueprint
Family: Hand-Drawn
Look: A drafted cyanotype: white line-work bleeding slightly into mottled, sun-faded Prussian-blue paper, orthographic front/side views with a hatched section, dimension chains, a zoned border, title block and worn fold creases.
Motion: Construction and projection lines sweep across first, object lines ink in view by view, then dimensions and lettering pop in as the camera slowly pans along the sheet.
How it explains: One large drafting sheet: a plotter head inks crisp white line-work and stencilled lettering while the camera tracks across and then down the sheet to the chart detail; the model is a gear-and-knob mechanism whose dials turn, arrows are drafted leaders, and numbers tick over in mono lettering inside dimension-style callouts.

H HELPER LIBRARY (lib/harness.js; the registerStyle/H.render parts are only needed by the original gallery)
```js
// Shared helpers + style registry for the style-frame explorations.
// Every style is a pure function of time: draw(ctx, W, H, t). Same t => same pixels.
(function () {
  const STYLES = (window.STYLES = window.STYLES || []);
  window.registerStyle = (s) => STYLES.push(s);

  const H = (window.H = {});

  // ---------- seeded randomness ----------
  // H.rng(seed) -> r() in [0,1). Extras: r.range(a,b), r.int(a,b), r.pick(arr), r.gauss(), r.chance(p)
  H.rng = function (seed = 1) {
    let a = typeof seed === 'string' ? hashStr(seed) : (seed >>> 0) || 1;
    const r = function () {
      a |= 0; a = (a + 0x6d2b79f5) | 0;
      let t = Math.imul(a ^ (a >>> 15), 1 | a);
      t = (t + Math.imul(t ^ (t >>> 7), 61 | t)) ^ t;
      return ((t ^ (t >>> 14)) >>> 0) / 4294967296;
    };
    r.range = (lo, hi) => lo + r() * (hi - lo);
```

```
    r.int = (lo, hi) => Math.floor(lo + r() * (hi - lo + 1));
    r.pick = (arr) => arr[Math.floor(r() * arr.length)];
    r.chance = (p) => r() < p;
    r.gauss = () => { let u = 0, v = 0; while (!u) u = r(); while (!v) v = r(); return Math.sqrt(-2 *
Math.log(u)) * Math.cos(2 * Math.PI * v); };
    return r;
  };
  function hashStr(s) { let h = 2166136261; for (let i = 0; i < s.length; i++) { h ^=
s.charCodeAt(i); h = Math.imul(h, 16777619); } return h >>> 0; }

  // ---------- gradient noise ----------
  // H.makeNoise(seed) -> { n2(x,y), n3(x,y,z), fbm2(x,y,oct), fbm3(x,y,z,oct) }  (outputs
roughly -1..1)
  H.makeNoise = function (seed = 1) {
    const r = H.rng(seed), p = new Uint8Array(512), perm = [...Array(256).keys()];
    for (let i = 255; i > 0; i--) { const j = Math.floor(r() * (i + 1)); [perm[i], perm[j]] = [perm[j],
perm[i]]; }
    for (let i = 0; i < 512; i++) p[i] = perm[i & 255];
    const fade = (t) => t * t * t * (t * (t * 6 - 15) + 10);
    const lerp = (a, b, t) => a + (b - a) * t;
    const g3 = (h, x, y, z) => { h &= 15; const u = h < 8 ? x : y, v = h < 4 ? y : h === 12 || h ===
14 ? x : z; return ((h & 1) ? -u : u) + ((h & 2) ? -v : v); };
    function n3(x, y, z) {
      const X = Math.floor(x) & 255, Y = Math.floor(y) & 255, Z = Math.floor(z) & 255;
      x -= Math.floor(x); y -= Math.floor(y); z -= Math.floor(z);
      const u = fade(x), v = fade(y), w = fade(z);
      const A = p[X] + Y, AA = p[A] + Z, AB = p[A + 1] + Z, B = p[X + 1] + Y, BA = p[B] + Z, BB
= p[B + 1] + Z;
      return lerp(lerp(lerp(g3(p[AA], x, y, z), g3(p[BA], x - 1, y, z), u), lerp(g3(p[AB], x, y - 1, z),
g3(p[BB], x - 1, y - 1, z), u), v),
        lerp(lerp(g3(p[AA + 1], x, y, z - 1), g3(p[BA + 1], x - 1, y, z - 1), u), lerp(g3(p[AB + 1], x, y
- 1, z - 1), g3(p[BB + 1], x - 1, y - 1, z - 1), u), v), w);
    }
    const n2 = (x, y) => n3(x, y, 0.5);
    const fbm3 = (x, y, z, oct = 4) => { let s = 0, a = 0.5, f = 1, norm = 0; for (let i = 0; i < oct;
i++) { s += a * n3(x * f, y * f, z * f); norm += a; a *= 0.5; f *= 2; } return s / norm; };
    const fbm2 = (x, y, oct = 4) => fbm3(x, y, 0.5, oct);
    return { n2, n3, fbm2, fbm3 };
  };

  // ---------- math ----------
  H.lerp = (a, b, t) => a + (b - a) * t;
  H.clamp = (v, lo = 0, hi = 1) => Math.max(lo, Math.min(hi, v));
  H.map = (v, a, b, c, d) => c + ((v - a) / (b - a)) * (d - c);
  H.smooth = (e0, e1, x) => { const t = H.clamp((x - e0) / (e1 - e0)); return t * t * (3 - 2 * t); };
  H.ease = {
    inOut: (t) => (t < 0.5 ? 4 * t * t * t : 1 - Math.pow(-2 * t + 2, 3) / 2),
    out: (t) => 1 - Math.pow(1 - t, 3),
```

```
    in: (t) => t * t * t,
    back: (t) => { const c = 1.70158; return 1 + (c + 1) * Math.pow(t - 1, 3) + c * Math.pow(t - 1,
2); },
  };

  // ---------- color ----------
  H.hex = (h) => { h = h.replace('#', ''); if (h.length === 3) h = h.split('').map((c) => c +
c).join(''); const n = parseInt(h, 16); return [(n >> 16) & 255, (n >> 8) & 255, n & 255]; };
  H.rgba = (hexOrArr, a = 1) => { const [r, g, b] = Array.isArray(hexOrArr) ? hexOrArr :
H.hex(hexOrArr); return `rgba(${r | 0},${g | 0},${b | 0},${a})`; };
  H.mix = (c1, c2, t) => { const a = Array.isArray(c1) ? c1 : H.hex(c1), b = Array.isArray(c2) ?
c2 : H.hex(c2); return [0, 1, 2].map((i) => a[i] + (b[i] - a[i]) * t); };

  // ---------- canvas utilities ----------
  // Offscreen canvas of the same logical size (use for layers, masks, compositing).
  H.layer = (W, H_, ) => { const c = document.createElement('canvas'); c.width = W; c.height
= H_; return c; };
  // Monochrome grain / film noise over the whole canvas. amount ~ 0.03-0.15
  H.grain = function (ctx, W, Hh, amount = 0.08, seed = 7, mono = true) {
    const img = ctx.getImageData(0, 0, ctx.canvas.width, ctx.canvas.height), d = img.data, r =
H.rng(seed), k = amount * 255;
    for (let i = 0; i < d.length; i += 4) {
      const n = (r() - 0.5) * k;
      if (mono) { d[i] += n; d[i + 1] += n; d[i + 2] += n; }
      else { d[i] += (r() - 0.5) * k; d[i + 1] += (r() - 0.5) * k; d[i + 2] += (r() - 0.5) * k; }
    }
    ctx.putImageData(img, 0, 0);
  };
  // Paper-ish texture tile: fills with base color + fibrous noise (cheap; drawn into ctx).
  H.paper = function (ctx, W, Hh, base = '#f3ecdc', seed = 3, strength = 0.06) {
    ctx.save(); ctx.fillStyle = base; ctx.fillRect(0, 0, W, Hh);
    const nz = H.makeNoise(seed), cell = 4, [br, bg, bb] = H.hex(base);
    for (let y = 0; y < Hh; y += cell) for (let x = 0; x < W; x += cell) {
      const v = nz.fbm2(x * 0.01, y * 0.01, 3) * 0.6 + nz.n2(x * 0.15, y * 0.04) * 0.4;
      const s = 1 + v * strength * 2;
      ctx.fillStyle = `rgb(${br * s | 0},${bg * s | 0},${bb * s | 0})`; ctx.fillRect(x, y, cell, cell);
    }
    ctx.restore();
  };
  // Vignette
  H.vignette = function (ctx, W, Hh, strength = 0.5, color = '0,0,0') {
    const g = ctx.createRadialGradient(W / 2, Hh / 2, Math.min(W, Hh) * 0.3, W / 2, Hh / 2,
Math.hypot(W, Hh) * 0.6);
    g.addColorStop(0, `rgba(${color},0)`); g.addColorStop(1, `rgba(${color},${strength})`);
    ctx.save(); ctx.fillStyle = g; ctx.fillRect(0, 0, W, Hh); ctx.restore();
  };

  // ---------- rendering entry ----------
```

```
  // Renders style into canvas at logical 1280x720 (canvas can be any pixel size; it is scaled).
  H.W = 1280; H.H = 720;
  H.loadFonts = async function (style) {
    const specs = style.fonts || [];
    try { const sample = 'AaZz09' + (window.FONT_SAMPLE || '');
      await Promise.all(specs.map((f) => document.fonts.load(f, sample))); await
document.fonts.ready; } catch (e) { /* render anyway */ }
  };
  H.render = function (style, canvas, t) {
    const ctx = canvas.getContext('2d', { willReadFrequently: !!style.readsPixels });
    const sx = canvas.width / H.W, sy = canvas.height / H.H;
    ctx.setTransform(1, 0, 0, 1, 0, 0); ctx.globalAlpha = 1; ctx.globalCompositeOperation =
'source-over'; ctx.filter = 'none';
    ctx.clearRect(0, 0, canvas.width, canvas.height);
    ctx.save(); ctx.scale(sx, sy);
    style.draw(ctx, H.W, H.H, t == null ? style.frameT || 0 : t);
    ctx.restore();
  };
})();
```

SOUND KIT (lib/sound.js)
```js
// sound.js: a small synthesized sound kit for the style explainers. Plain Web Audio, no audio
files.
// Voiceover mode: SOUND.kit(ctx, { voiceover: true }) drops beds and music, plays anything
marked minor
// (play(..., { minor: true }), tone/noise { minor: true }, or scheduled inside k.withMinor(() =>
...)) at about 60%
// volume instead of full, and sits all effects a little lower, so the narration stays on top while
the piece still
// sounds alive. In the full mix, minor sounds play at full volume. Scores can read k.vo to
branch further.
// A style adds   explainerSound(k) { k.bed('room', 0, D); k.play('pop', 1.25); ... }   and the
same
// call schedules into a live AudioContext (the player) or an OfflineAudioContext (the video
render).
// Times are seconds on the explainer's own clock. Randomness is seeded, so every render
is identical.
(function () {
  'use strict';
  const SOUND = (window.SOUND = {});
  const TINY = 1e-4;

  // seeded random in [0,1), keyed by any string
  function rand(key) {
    let h = 2166136261;
    for (let i = 0; i < key.length; i++) { h ^= key.charCodeAt(i); h = Math.imul(h, 16777619); }
```

```
    let a = h >>> 0 || 1;
    return () => { a = (a + 0x6d2b79f5) | 0; let t = Math.imul(a ^ (a >>> 15), 1 | a); t = (t +
Math.imul(t ^ (t >>> 7), 61 | t)) ^ t; return ((t ^ (t >>> 14)) >>> 0) / 4294967296; };
  }

  // note names to Hz: 'A4' = 440, 'C#5', 'Eb3'
  const STEP = { C: 0, D: 2, E: 4, F: 5, G: 7, A: 9, B: 11 };
  function hz(n) {
    if (typeof n === 'number') return n;
    const m = /^([A-G])([#b]?)(-?\d)$/.exec(n);
    const semis = STEP[m[1]] + (m[2] === '#' ? 1 : m[2] === 'b' ? -1 : 0) + (Number(m[3]) - 4) *
12 - 9;
    return 440 * Math.pow(2, semis / 12);
  }
  SOUND.hz = hz;

  SOUND.kit = function (ctx, opts) {
    const o = opts || {};
    const VO = !!o.voiceover;
    const MINOR = 0.6; // voiceover-mode volume for minor sounds
    let inMinor = false;
    const gainOf = (p, d) => (p.gain == null ? d : p.gain) * (VO && (p.minor || inMinor) ?
MINOR : 1);
    const T0 = (o.start || 0) - 0; // explainer time that maps to ctx time `base`
    const base = ctx.currentTime + (o.delay || 0);
    // Start times snap to the sample grid (+ a quarter sample): in Chrome a source started a
hair off the grid can
    // spike to about twice its set level.
    const SR = ctx.sampleRate;
    const at = (t) => (Math.round((base + (t - T0)) * SR) + 0.25) / SR;

    // ---- buses: sfx, amb, music -> compressor -> master; one shared reverb send
    const master = ctx.createGain(); master.gain.value = o.master == null ? 0.9 : o.master;
    const comp = ctx.createDynamicsCompressor();
    comp.threshold.value = -16; comp.knee.value = 8; comp.ratio.value = 3;
comp.attack.value = 0.004; comp.release.value = 0.18;
    comp.connect(master); master.connect(o.dest || ctx.destination);
    const bus = {};
    for (const [name, g] of Object.entries({ sfx: VO ? 0.7 : 0.85, amb: 0.5, music: 0.45 })) {
bus[name] = ctx.createGain(); bus[name].gain.value = g; bus[name].connect(comp); }

    // generated room reverb: decaying stereo noise, darker toward the tail
    const verb = ctx.createConvolver();
    {
      const secs = 1.6, n = Math.floor(secs * ctx.sampleRate), ir = ctx.createBuffer(2, n,
ctx.sampleRate);
      for (let c = 0; c < 2; c++) {
        const r = rand('verb' + c), d = ir.getChannelData(c); let lp = 0;
```

```
      for (let i = 0; i < n; i++) { const u = i / n; lp += (0.6 - 0.45 * u) * ((r() * 2 - 1) - lp); d[i] = lp *
Math.pow(1 - u, 3); }
    }
    verb.buffer = ir;
  }
  const verbIn = ctx.createGain(); verbIn.gain.value = 0.9; verbIn.connect(verb);
verb.connect(comp);

  // seeded noise buffers: white, pink-ish and brown
  const noiseBuf = {};
  {
    const n = Math.floor(2.5 * ctx.sampleRate);
    for (const kind of ['white', 'pink', 'brown']) {
      const b = ctx.createBuffer(1, n, ctx.sampleRate), d = b.getChannelData(0), r =
rand('noise-' + kind);
      let p0 = 0, p1 = 0, p2 = 0, br = 0;
      for (let i = 0; i < n; i++) {
        const w = r() * 2 - 1;
        if (kind === 'white') d[i] = w;
        else if (kind === 'pink') { p0 = 0.997 * p0 + 0.029 * w; p1 = 0.985 * p1 + 0.032 * w; p2 =
0.95 * p2 + 0.048 * w; d[i] = (p0 + p1 + p2 + w * 0.05) * 2.2; }
        else { br = (br + 0.02 * w) / 1.02; d[i] = br * 3.2; }
      }
      noiseBuf[kind] = b;
    }
  }

  // ---- automation helpers
  // a value or [[dt, value], ...] from the voice start; ramps are exponential (frequencies)
  function sweep(param, v, t) {
    if (!Array.isArray(v)) { param.setValueAtTime(v, t); return; }
    param.setValueAtTime(v[0][1], t + v[0][0]);
    for (let i = 1; i < v.length; i++) param.exponentialRampToValueAtTime(Math.max(v[i][1],
1), t + v[i][0]);
  }
  // amplitude: linear attack to `gain`, optional hold, exponential release to silence by `dur`
  function shape(param, t, gain, dur, a, hold) {
    a = Math.min(a == null ? 0.004 : a, dur * 0.9);
    hold = Math.min(hold || 0, dur - a);
    param.setValueAtTime(0, t);
    param.linearRampToValueAtTime(gain, t + a);
    if (hold > 0) param.setValueAtTime(gain, t + a + hold);
    param.exponentialRampToValueAtTime(TINY, t + dur);
    param.setValueAtTime(0, t + dur + 0.001);
  }
  function out(node, p) {
    const pan = ctx.createStereoPanner(); pan.pan.value = Math.max(-1, Math.min(1, p.pan
|| 0));
```

```
    node.connect(pan); pan.connect(bus[p.bus || 'sfx']);
    if (p.verb) { const s = ctx.createGain(); s.gain.value = p.verb; pan.connect(s);
s.connect(verbIn); }
  }
  function filterNode(p, t) {
    if (!p.filter) return null;
    const f = ctx.createBiquadFilter(); f.type = p.filter; f.Q.value = p.q == null ? 1 : p.q;
    sweep(f.frequency, p.f == null ? 1000 : p.f, t);
    return f;
  }

  // ---- primitives
  // tone(t, { f, type, dur, gain, a, hold, pan, verb, bus, filter, f2 (filter freq), q, detune })
  function tone(time, p) {
    const t = at(time), dur = p.dur || 0.2;
    if (t + dur < ctx.currentTime || (time < T0 - 0.001 && dur < 1.5)) return;
    const osc = ctx.createOscillator(); osc.type = p.type || 'sine';
    sweep(osc.frequency, Array.isArray(p.f) ? p.f.map(([d, v]) => [d, hz(v)]) : hz(p.f || 440), t);
    if (p.detune) osc.detune.value = p.detune;
    const g = ctx.createGain(); shape(g.gain, t, gainOf(p, 0.3), dur, p.a, p.hold);
    let node = osc;
    if (p.filter) { const f = ctx.createBiquadFilter(); f.type = p.filter; f.Q.value = p.q == null ? 0.8
: p.q; sweep(f.frequency, p.f2 || 2000, t); osc.connect(f); node = f; }
    node.connect(g); out(g, p);
    osc.start(t); osc.stop(t + dur + 0.02);
  }
  // noise(t, { dur, color, filter, f, q, gain, a, hold, pan, verb, bus, rate })
  function noise(time, p) {
    const t = at(time), dur = p.dur || 0.2;
    if (t + dur < ctx.currentTime || (time < T0 - 0.001 && dur < 1.5)) return;
    const src = ctx.createBufferSource(); src.buffer = noiseBuf[p.color || 'white']; src.loop =
true;
    if (p.rate) src.playbackRate.value = p.rate;
    const g = ctx.createGain(); shape(g.gain, t, gainOf(p, 0.3), dur, p.a, p.hold);
    const f = filterNode(p, t);
    if (f) { src.connect(f); f.connect(g); } else src.connect(g);
    out(g, p);
    const off = rand('off' + time + (p.f || ''))() * 2;
    src.start(t, off); src.stop(t + dur + 0.02);
  }

  // ---- named sounds. play(name, t, opts): opts.gain scales the whole sound, opts.pan
places it.
  const S = {
    // UI / motion graphics
    pop: (t, o) => { tone(t, { f: [[0, o.f || 620], [0.07, (o.f || 620) * 0.45]], dur: 0.1, gain: 0.32 *
o.g, pan: o.pan }); noise(t, { dur: 0.012, filter: 'highpass', f: 3000, gain: 0.12 * o.g, pan: o.pan
}); },
```

```
    tick: (t, o) => noise(t, { dur: 0.018, filter: 'bandpass', f: o.f || 4200, q: 2.5, gain: 0.3 * o.g, pan: o.pan }),
    click: (t, o) => { tone(t, { f: o.f || 1900, type: 'square', dur: 0.02, gain: 0.08 * o.g, filter: 'lowpass', f2: 3000, pan: o.pan }); noise(t, { dur: 0.01, filter: 'highpass', f: 2500, gain: 0.15 * o.g, pan: o.pan }); },
    tock: (t, o) => { tone(t, { f: o.f || 820, dur: 0.07, gain: 0.25 * o.g, pan: o.pan }); noise(t, { dur: 0.05, filter: 'bandpass', f: o.f || 820, q: 9, gain: 0.35 * o.g, pan: o.pan }); },
    blip: (t, o) => tone(t, { f: o.f || 'E6', type: o.type || 'square', dur: o.dur || 0.07, gain: 0.07 * o.g, a: 0.002, hold: (o.dur || 0.07) * 0.6, pan: o.pan }),
    whoosh: (t, o) => { const d = o.dur || 0.45; noise(t, { dur: d, color: 'pink', filter: 'bandpass', f: [[0, 350], [d * 0.55, o.top || 2600], [d, 700]], q: 0.9, gain: 0.32 * o.g, a: d * 0.5, pan: o.pan, verb: 0.15 }); },
    swish: (t, o) => { const d = o.dur || 0.18; noise(t, { dur: d, filter: 'highpass', f: [[0, 1800], [d, 5200]], q: 0.7, gain: 0.22 * o.g, a: d * 0.35, pan: o.pan }); },
    riser: (t, o) => { const d = o.dur || 0.8; noise(t, { dur: d, color: 'pink', filter: 'bandpass', f: [[0, 300], [d, 4000]], q: 1.2, gain: 0.25 * o.g, a: d * 0.95, pan: o.pan, verb: 0.2 }); tone(t, { f: [[0, 180], [d, 720]], type: 'triangle', dur: d, gain: 0.06 * o.g, a: d * 0.9 }); },
    thud: (t, o) => { tone(t, { f: [[0, o.f || 130], [0.16, 42]], dur: 0.26, gain: 0.55 * o.g, pan: o.pan }); noise(t, { dur: 0.09, color: 'brown', filter: 'lowpass', f: 500, gain: 0.35 * o.g, pan: o.pan }); },
    impact: (t, o) => { S.thud(t, o); noise(t, { dur: 0.7, color: 'pink', filter: 'lowpass', f: [[0, 2400], [0.6, 300]], gain: 0.3 * o.g, verb: 0.35, pan: o.pan }); },
    stamp: (t, o) => { S.thud(t, { ...o, f: 160 }); noise(t, { dur: 0.05, filter: 'bandpass', f: 1300, q: 1.2, gain: 0.4 * o.g, pan: o.pan }); noise(t + 0.004, { dur: 0.012, filter: 'highpass', f: 3500, gain: 0.25 * o.g, pan: o.pan }); },
    ding: (t, o) => { const f = hz(o.f || 'E6'); tone(t, { f, dur: 1.3, gain: 0.2 * o.g, verb: 0.3, pan: o.pan }); tone(t, { f: f * 2.76, dur: 0.5, gain: 0.05 * o.g, verb: 0.3, pan: o.pan }); },
    success: (t, o) => { S.ding(t, { ...o, f: o.f || 'C6' }); S.ding(t + 0.09, { ...o, f: hz(o.f || 'C6') * 1.5, g: o.g * 0.9 }); },
    error: (t, o) => { tone(t, { f: o.f || 'A3', type: 'square', dur: 0.13, gain: 0.09 * o.g, hold: 0.08, filter: 'lowpass', f2: 1400, pan: o.pan }); tone(t + 0.15, { f: hz(o.f || 'A3') * 0.8, type: 'square', dur: 0.22, gain: 0.09 * o.g, hold: 0.14, filter: 'lowpass', f2: 1200, pan: o.pan }); },
    buzz: (t, o) => { const d = o.dur || 0.3; tone(t, { f: o.f || 110, type: 'sawtooth', dur: d, gain: 0.12 * o.g, hold: d * 0.7, filter: 'lowpass', f2: 900, pan: o.pan }); tone(t, { f: (o.f || 110) * 1.01, type: 'sawtooth', dur: d, gain: 0.1 * o.g, hold: d * 0.7, filter: 'lowpass', f2: 900, pan: o.pan }); },
    sparkle: (t, o) => { const r = rand('sparkle' + t), notes = ['C6', 'D6', 'E6', 'G6', 'A6', 'C7']; for (let i = 0; i < (o.n || 6); i++) { const ti = t + i * 0.04 + r() * 0.01, f = hz(notes[Math.floor(r() * notes.length)]), pan = r() * 1.2 - 0.6; tone(ti, { f, dur: 0.6, gain: 0.04 * o.g, a: 0.002, verb: 0.35, pan }); tone(ti, { f: f * 2.76, dur: 0.2, gain: 0.012 * o.g, a: 0.002, pan }); } },
    glitch: (t, o) => { const r = rand('glitch' + t); for (let i = 0; i < 7; i++) tone(t + i * 0.022, { f: 200 + r() * 1800, type: 'square', dur: 0.02, gain: 0.07 * o.g, a: 0.001, hold: 0.015, pan: r() * 1.4 - 0.7 }); noise(t, { dur: 0.16, filter: 'bandpass', f: 2500, q: 0.6, gain: 0.12 * o.g }); },
    shutter: (t, o) => { S.click(t, o); noise(t + 0.06, { dur: 0.04, filter: 'bandpass', f: 2200, q: 1, gain: 0.2 * o.g, pan: o.pan }); },
    count: (t, o) => S.tick(t, { ...o, f: o.f || 5200, g: o.g * 0.7 }),

    // hand-made media
    // stroke: a pen/marker/chalk/pencil stroke over `dur` seconds. material sets the surface sound.
```

```
  stroke: (t, o) => {
    const d = o.dur || 0.4, m = o.material || 'marker', r = rand('stroke' + m + t);
    const M = {
      marker: { f: 2900, q: 2.2, g: 0.16, grain: 0.05 },
      chalk: { f: 1700, q: 0.6, g: 0.22, grain: 0.028 },
      pencil: { f: 4600, q: 0.8, g: 0.16, grain: 0.04, filter: 'highpass' },
      pen: { f: 4200, q: 1.6, g: 0.08, grain: 0.06 },
      brush: { f: 900, q: 0.5, g: 0.1, grain: 0.09, color: 'pink' },
      sharpie: { f: 2400, q: 1.4, g: 0.2, grain: 0.045 },
    }[m];
    for (let x = 0; x < d; x += M.grain * (0.7 + r() * 0.6)) {
      const gd = M.grain * (0.8 + r() * 0.9);
      noise(t + x, { dur: gd, color: M.color, filter: M.filter || 'bandpass', f: M.f * (0.8 + r() * 0.4),
q: M.q, gain: M.g * o.g * (0.6 + r() * 0.5), a: gd * 0.3, pan: (o.pan || 0) + (r() - 0.5) * 0.2 });
    }
  },
  // paper: picking up / slapping down / turning a sheet
  paper: (t, o) => { const d = o.dur || 0.35, r = rand('paper' + t); for (let x = 0; x < d; x +=
0.025 + r() * 0.03) noise(t + x, { dur: 0.05, filter: 'bandpass', f: 1200 + r() * 2600, q: 0.9, gain:
0.12 * o.g * (1 - x / d * 0.5), pan: o.pan }); },
  slap: (t, o) => { noise(t, { dur: 0.07, filter: 'highpass', f: 900, gain: 0.45 * o.g, a: 0.001, pan:
o.pan }); tone(t, { f: [[0, 180], [0.06, 70]], dur: 0.1, gain: 0.25 * o.g, pan: o.pan }); },
  tape: (t, o) => { noise(t, { dur: o.dur || 0.5, filter: 'bandpass', f: [[0, 900], [o.dur || 0.5,
2200]], q: 1.8, gain: 0.18 * o.g, a: 0.02, pan: o.pan }); },
  press: (t, o) => { S.thud(t, { ...o, f: 95, g: o.g * 0.9 }); noise(t + 0.01, { dur: 0.25, color:
'brown', filter: 'lowpass', f: 700, gain: 0.25 * o.g, pan: o.pan }); },
  roller: (t, o) => { const d = o.dur || 0.6; noise(t, { dur: d, color: 'brown', filter: 'bandpass', f:
380, q: 1.5, gain: 0.3 * o.g, a: 0.05, hold: d * 0.6, pan: o.pan }); for (let x = 0; x < d; x += 0.07)
S.tock(t + x, { f: 300, g: o.g * 0.15, pan: o.pan }); },
  erase: (t, o) => { const d = o.dur || 0.6, r = rand('erase' + t); for (let x = 0; x < d; x += 0.06
+ r() * 0.04) noise(t + x, { dur: 0.08, color: 'pink', filter: 'bandpass', f: 700 + r() * 500, q: 0.8,
gain: 0.22 * o.g, a: 0.03, pan: (r() - 0.5) * 0.5 }); },

  // machines and screens
  key: (t, o) => { const r = rand('key' + t); noise(t, { dur: 0.025, filter: 'bandpass', f: 2600 + r()
* 1200, q: 2, gain: 0.25 * o.g, a: 0.001, pan: o.pan }); tone(t, { f: 160 + r() * 40, dur: 0.04, gain:
0.12 * o.g, pan: o.pan }); },
  typing: (t, o) => { const r = rand('typing' + t), n = o.n || 8, rate = o.rate || 0.09; for (let i = 0;
i < n; i++) S.key(t + i * rate * (0.75 + r() * 0.5), { g: o.g * (0.7 + r() * 0.4), pan: o.pan }); },
  typebar: (t, o) => { S.key(t, o); noise(t + 0.008, { dur: 0.03, filter: 'bandpass', f: 1100, q: 4,
gain: 0.35 * o.g, pan: o.pan }); },
  bell: (t, o) => S.ding(t, { ...o, f: o.f || 'A6' }),
  boot: (t, o) => { tone(t, { f: [[0, 80], [0.35, 1600]], type: 'sine', dur: 0.5, gain: 0.08 * o.g, a:
0.3 }); S.blip(t + 0.45, { ...o, f: 'C6', dur: 0.12 }); },
  crtOn: (t, o) => { noise(t, { dur: 0.25, filter: 'highpass', f: 2000, gain: 0.2 * o.g, a: 0.001 });
tone(t, { f: 15700, dur: 1.5, gain: 0.02 * o.g, a: 0.1, hold: 1 }); S.thud(t, { ...o, f: 90, g: o.g * 0.5
}); },
```

```
    neon: (t, o) => { const d = o.dur || 0.25, r = rand('neon' + t); for (let x = 0; x < d; x += 0.035 + r() * 0.04) tone(t + x, { f: 120, type: 'sawtooth', dur: 0.03, gain: 0.3 * o.g * (0.5 + r() * 0.5), filter: 'bandpass', f2: 1200, q: 1.2, pan: o.pan }); },
    // chip (8-bit console) sounds: raw square/triangle, no filtering
    coin: (t, o) => { tone(t, { f: 'B5', type: 'square', dur: 0.07, gain: 0.07 * o.g, a: 0.001, hold: 0.06 }); tone(t + 0.07, { f: 'E6', type: 'square', dur: 0.3, gain: 0.07 * o.g, a: 0.001, hold: 0.1 }); },
    jump: (t, o) => tone(t, { f: [[0, 260], [0.16, 820]], type: 'square', dur: 0.18, gain: 0.07 * o.g, a: 0.001, hold: 0.12 }),
    hurt: (t, o) => { tone(t, { f: [[0, 600], [0.25, 90]], type: 'square', dur: 0.28, gain: 0.08 * o.g, a: 0.001, hold: 0.2 }); noise(t, { dur: 0.2, filter: 'lowpass', f: 1800, gain: 0.15 * o.g, rate: 0.25 }); },
    powerup: (t, o) => ['C5', 'E5', 'G5', 'C6', 'E6', 'G6', 'C7'].forEach((n, i) => tone(t + i * 0.045, { f: n, type: 'square', dur: 0.06, gain: 0.06 * o.g, a: 0.001, hold: 0.05 })),
    select: (t, o) => { S.blip(t, { ...o, f: 'A5', dur: 0.04 }); S.blip(t + 0.05, { ...o, f: 'E6', dur: 0.05 }); },
    text: (t, o) => { const n = o.n || 10, rate = o.rate || 0.05; for (let i = 0; i < n; i++) tone(t + i * rate, { f: i % 2 ? 'C6' : 'G5', type: 'square', dur: 0.025, gain: 0.04 * o.g, a: 0.001, hold: 0.015 }); },
  };

  function play(name, t, opts) {
    const fn = S[name];
    if (!fn) throw new Error('unknown sound: ' + name);
    const o2 = Object.assign({}, opts); o2.g = o2.gain == null ? 1 : o2.gain;
    if (opts && opts.minor) withMinor(() => fn(t, o2)); else fn(t, o2);
  }

  // everything scheduled inside fn counts as minor (quieter in voiceover mode)
  function withMinor(fn) { const was = inMinor; inMinor = true; try { fn(); } finally { inMinor = was; } }

  // ---- beds: continuous sound under a stretch of time. bed(name, t0, t1, opts)
  function bed(name, t0, t1, p) {
    if (VO) return;
    p = p || {}; const g = p.gain == null ? 1 : p.gain, d = t1 - t0, fade = Math.min(0.4, d / 4);
    const long = (q) => Object.assign({ dur: d, a: fade, hold: d - fade * 2, bus: 'amb' }, q);
    if (name === 'room') noise(t0, long({ color: 'brown', filter: 'lowpass', f: 320, gain: 0.12 * g }));
    else if (name === 'hiss') noise(t0, long({ color: 'pink', filter: 'highpass', f: 5000, gain: 0.02 * g }));
    else if (name === 'hum') { tone(t0, long({ f: 60, gain: 0.05 * g })); tone(t0, long({ f: 120, gain: 0.025 * g })); tone(t0, long({ f: 180, type: 'triangle', gain: 0.012 * g })); }
    else if (name === 'rain') { noise(t0, long({ color: 'pink', filter: 'bandpass', f: 2500, q: 0.4, gain: 0.08 * g })); const r = rand('rain'); for (let x = 0; x < d; x += 0.04 + r() * 0.08) noise(t0 + x, { dur: 0.01, filter: 'highpass', f: 4000, gain: 0.04 * g * r(), bus: 'amb', pan: r() * 1.6 - 0.8 }); }
    else if (name === 'projector') { noise(t0, long({ color: 'pink', filter: 'highpass', f: 3000, gain: 0.03 * g })); for (let x = 0; x < d; x += 1 / 24) S.tock(t0 + x, { f: 700, g: 0.04 * g, pan: 0.3 }); }
```

```
    else if (name === 'vinyl') { noise(t0, long({ color: 'pink', filter: 'bandpass', f: 1500, q: 0.5, gain: 0.025 * g })); const r = rand('vinyl'); for (let x = 0; x < d; x += 0.08 + r() * 0.4) noise(t0 + x, { dur: 0.006, filter: 'highpass', f: 2500, gain: 0.08 * g * r(), bus: 'amb' }); }
    else throw new Error('unknown bed: ' + name);
  }

  // ---- music: optional light beds that sit under the effects. music(name, t0, t1, { bpm, chords, gain })
  // chords: list of note arrays, one per bar, cycled. e.g. [['C4','E4','G4','B4'], ['A3','C4','E4','G4']]
  function music(name, t0, t1, p) {
    if (VO) return;
    p = p || {}; const g = p.gain == null ? 1 : p.gain, bpm = p.bpm || 100, beat = 60 / bpm, bar = beat * 4;
    const chords = p.chords || [['C4', 'E4', 'G4', 'B4'], ['A3', 'C4', 'E4', 'G4'], ['F3', 'A3', 'C4', 'E4'], ['G3', 'B3', 'D4', 'F4']];
    let i = 0;
    for (let t = t0; t < t1 - 0.05; t += bar, i++) {
      const ch = chords[i % chords.length], len = Math.min(bar, t1 - t);
      if (name === 'pad' || name === 'pulse') ch.forEach((n, k) => { tone(t, { f: n, type: 'triangle', dur: len + 0.3, gain: 0.035 * g, a: 0.25, hold: len - 0.3, bus: 'music', filter: 'lowpass', f2: 1400, pan: (k - 1.5) * 0.25, verb: 0.25 }); tone(t, { f: hz(n) * 1.004, type: 'sine', dur: len + 0.3, gain: 0.03 * g, a: 0.3, hold: len - 0.3, bus: 'music' }); });
      if (name === 'pulse') for (let b = 0; b < 4 && t + b * beat < t1 - 0.05; b++) {
        const tb = t + b * beat;
        tone(tb, { f: [[0, 110], [0.1, 50]], dur: 0.18, gain: (b % 2 ? 0.16 : 0.26) * g, bus: 'music' });
        noise(tb + beat / 2, { dur: 0.04, filter: 'highpass', f: 7000, gain: 0.05 * g, bus: 'music' });
        tone(tb + beat / 2, { f: hz(ch[0]) / 2, type: 'triangle', dur: beat * 0.45, gain: 0.08 * g, bus: 'music' });
      }
      if (name === 'chip') for (let s = 0; s < 8 && t + s * beat / 2 < t1 - 0.05; s++) {
        const ts = t + s * beat / 2;
        tone(ts, { f: hz(ch[0]) / 2, type: 'triangle', dur: beat * 0.45, gain: 0.12 * g, a: 0.001, hold: beat * 0.35, bus: 'music' });
        tone(ts, { f: ch[[0, 1, 2, 3, 2, 1, 2, 3][s]], type: 'square', dur: beat * 0.4, gain: 0.03 * g, a: 0.001, hold: beat * 0.25, bus: 'music' });
      }
    }
  }

  return { play, bed, music, tone, noise, withMinor, hz, vo: VO, sounds: Object.keys(S) };
};

// Live playback for a player: SOUND.live(scoreFn, fromT, { voiceover, master }) starts the score from explainer time
// fromT right now and returns stop(). Call stop() on pause or seek, and SOUND.live again on play.
SOUND.live = function (fn, fromT, opts) {
```

```
    const ctx = new (window.AudioContext || window.webkitAudioContext)();
    fn(SOUND.kit(ctx, Object.assign({}, opts, { start: fromT || 0, delay: 0.03 })));
    return () => { try { ctx.close(); } catch (e) {} };
  };

  // Render a style's explainerSound into an OfflineAudioContext; returns interleaved float
stereo.
  SOUND.renderOffline = async function (fn, duration, sampleRate, kitOpts) {
    const sr = sampleRate || 48000, len = Math.ceil(duration * sr);
    const ctx = new OfflineAudioContext(2, len, sr);
    fn(SOUND.kit(ctx, kitOpts || {}));
    const buf = await ctx.startRendering();
    const L = buf.getChannelData(0), R = buf.getChannelData(1), inter = new
Float32Array(len * 2);
    let peak = 0;
    for (let i = 0; i < len; i++) { inter[2 * i] = L[i]; inter[2 * i + 1] = R[i]; peak = Math.max(peak,
Math.abs(L[i]), Math.abs(R[i])); }
    return { inter, peak, frames: len, sr };
  };
})();
```

REFERENCE STYLE SOURCE (styles/blueprint.js; W/H_ are always 1280x720)
```js
// Blueprint — cyanotype engineering drawing of a service robot.
{
  // >>>XE-ENGINE
  // ====================== explainer story engine (shared script: "How does AI learn?")
======================
  // Builds the diagram as timed world-space strokes/texts; each style renders them with its
own medium.
  const XC = {}; // cache for static textures (output still depends only on t)
  const XE = (() => {
    const clamp = (v, a = 0, b = 1) => Math.max(a, Math.min(b, v));
    const win = (t, a, b) => clamp((t - a) / (b - a));
    const eio = (x) => (x < 0.5 ? 4 * x * x * x : 1 - Math.pow(-2 * x + 2, 3) / 2);
    const camAt = (keys, t) => {
      if (t <= keys[0][0]) return keys[0].slice(1);
      for (let i = 0; i < keys.length - 1; i++) {
        const a = keys[i], b = keys[i + 1];
        if (t <= b[0]) {
          const u = eio((t - a[0]) / Math.max(1e-6, b[0] - a[0]));
          return [a[1] + (b[1] - a[1]) * u, a[2] + (b[2] - a[2]) * u, Math.exp(Math.log(a[3]) +
(Math.log(b[3]) - Math.log(a[3])) * u)];
        }
      }
      return keys[keys.length - 1].slice(1);
    };
```

```
  const circ = (x, y, rx, ry, a0 = 0, a1 = Math.PI * 2, n = 0) => {
    n = n || Math.max(12, Math.ceil(Math.abs(a1 - a0) * Math.max(rx, ry) / 6)); const o = [];
    for (let i = 0; i <= n; i++) { const a = a0 + (a1 - a0) * i / n; o.push([x + Math.cos(a) * rx, y + Math.sin(a) * ry]); }
    return o;
  };
  const quad = (p0, p1, p2, n = 30) => { const o = []; for (let i = 0; i <= n; i++) { const t = i / n, m = 1 - t; o.push([m * m * p0[0] + 2 * m * t * p1[0] + t * t * p2[0], m * m * p0[1] + 2 * m * t * p1[1] + t * t * p2[1]]); } return o; };
  const cubic = (p0, p1, p2, p3, n = 40) => { const o = []; for (let i = 0; i <= n; i++) { const t = i / n, m = 1 - t; o.push([m * m * m * p0[0] + 3 * m * m * t * p1[0] + 3 * m * t * t * p2[0] + t * t * t * p3[0], m * m * m * p0[1] + 3 * m * m * t * p1[1] + 3 * m * t * t * p2[1] + t * t * t * p3[1]]); } return o; };
  const rrect = (x, y, w, h, r) => {
    const o = [[x + r, y], [x + w - r, y]]; o.push(...circ(x + w - r, y + r, r, r, -Math.PI / 2, 0, 5).slice(1));
    o.push([x + w, y + h - r]); o.push(...circ(x + w - r, y + h - r, r, r, 0, Math.PI / 2, 5).slice(1));
    o.push([x + r, y + h]); o.push(...circ(x + r, y + h - r, r, r, Math.PI / 2, Math.PI, 5).slice(1));
    o.push([x, y + r]); o.push(...circ(x + r, y + r, r, r, Math.PI, Math.PI * 1.5, 5).slice(1));
    o.push([x + r + 6, y - 1]); return o;
  };
  const head = (P, len = 20, spread = 0.5) => {
    const a = P[P.length - 1], b = P[Math.max(0, P.length - 4)], an = Math.atan2(a[1] - b[1], a[0] - b[0]);
    return [[a[0] - Math.cos(an - spread) * len, a[1] - Math.sin(an - spread) * len], a, [a[0] - Math.cos(an + spread) * len, a[1] - Math.sin(an + spread) * len]];
  };
  const cat = (x, y, s) => [
    circ(x, y + 0.06 * s, 0.3 * s, 0.27 * s, -1.2, 5.1),
    [[x - 0.27 * s, y - 0.02 * s], [x - 0.24 * s, y - 0.34 * s], [x - 0.07 * s, y - 0.2 * s]],
    [[x + 0.07 * s, y - 0.2 * s], [x + 0.24 * s, y - 0.34 * s], [x + 0.27 * s, y - 0.02 * s]],
    [[x - 0.03 * s, y + 0.1 * s], [x, y + 0.13 * s], [x + 0.03 * s, y + 0.1 * s]],
    [[x - 0.12 * s, y + 0.12 * s], [x - 0.36 * s, y + 0.08 * s]], [[x - 0.12 * s, y + 0.16 * s], [x - 0.35 * s, y + 0.2 * s]],
    [[x + 0.12 * s, y + 0.12 * s], [x + 0.36 * s, y + 0.08 * s]], [[x + 0.12 * s, y + 0.16 * s], [x + 0.35 * s, y + 0.2 * s]],
  ];
  const dog = (x, y, s) => [
    circ(x, y + 0.04 * s, 0.26 * s, 0.28 * s, -1.4, 4.9),
    circ(x - 0.29 * s, y + 0.04 * s, 0.09 * s, 0.2 * s, 0, 6.4),
    circ(x + 0.29 * s, y + 0.04 * s, 0.09 * s, 0.2 * s, 0, 6.4),
    circ(x, y + 0.15 * s, 0.07 * s, 0.045 * s, 0, 6.4),
    [[x - 0.08 * s, y + 0.23 * s], [x, y + 0.26 * s], [x + 0.08 * s, y + 0.23 * s]],
  ];
  const eyes = (x, y, s, dy = 0.02) => [[x - 0.1 * s, y + dy * s], [x + 0.1 * s, y + dy * s]];
  const fmt = (n) => Math.round(n).toString().replace(/\B(?=(\d{3})+(?!\d))/g, ',');

  // L: layout options. mode: 'same' (progress drawn where the loop was) | 'below' | 'right'
```

```
  // model(cx, cy, adj, add): style supplies its model drawing; adj 0..1 = how far the knobs have been adjusted.
  const story = (t, L) => {
    const items = [];
    const line = (pts, role, w, a, b, g) => items.push({ k: 'line', pts, role, w, a, b, g });
    const text = (s, x, y, size, font, role, a, b, opt = {}) => items.push({ k: 'text', s, x, y, size, font, role, a, b, rot: opt.rot || 0, align: opt.align || 'left', g: opt.g });
    const dot = (x, y, r, role, a, g) => items.push({ k: 'dot', x, y, r, role, a, b: a + 0.05, g });
    const S = L.s || 1;
    // ---- 1 HOOK ----
    const TX = L.titleX ?? 640, TY = L.titleY ?? 350;
    text('How does AI learn?', TX, TY, 84 * S, 'title', 'ink', 0.15, 1.3, { align: 'center', rot: -0.015, g: 'A' });
    line([[TX - 330 * S, TY + 32 * S], [TX + 330 * S, TY + 24 * S]], 'acc1', 7, 1.3, 1.55, 'A');
    // ---- 2 DATA ----
    const MX = L.modelX ?? 2250, MY = L.modelY ?? 640;
    const mAdj = eio(win(t, 8.95, 9.5));
    L.model(MX, MY, mAdj, (it) => { it.g = it.g || 'M'; items.push(it); }, { line, text, dot }, t);
    text(L.modelName || 'model', MX, MY + 228, 38, 'label', 'ink', 3.15, 3.4, { align: 'center', g: 'M' });
    const CS = 118, cx0 = MX - 870, cy0 = MY - 175;
    for (let i = 0; i < 6; i++) {
      const col = i % 3, row = (i / 3) | 0, x = cx0 + col * 150 + (row ? 40 : 0), y = cy0 + row * 170, isCat = [1, 0, 1, 1, 0, 0][i];
      const a = 3.0 + i * 0.17;
      line(rrect(x - CS / 2, y - CS / 2, CS, CS, 12), 'ink', 4, a, a + 0.12, 'B');
      (isCat ? cat : dog)(x, y - 6, CS * 0.82).forEach((p) => line(p, 'ink', 3.2, a + 0.1, a + 0.22, 'B'));
      eyes(x, y - 6, CS * 0.82).forEach(([ex, ey]) => dot(ex, ey, 3.2, 'ink', a + 0.22, 'B'));
      // label tag hanging off the card
      const tx = x + 18, ty = y + CS / 2 - 4;
      line([[tx, ty], [tx + 70, ty], [tx + 84, ty + 19], [tx + 70, ty + 38], [tx, ty + 38], [tx, ty]], isCat ? 'acc1' : 'acc3', 3, a + 0.2, a + 0.3, 'B');
      text(isCat ? 'cat' : 'dog', tx + 8, ty + 30, 31, 'label', isCat ? 'acc1' : 'acc3', a + 0.28, a + 0.36, { g: 'B' });
    }
    // flow arrow into the model
    const fa = cubic([cx0 + 370, MY - 10], [cx0 + 470, MY - 60], [MX - 260, MY - 40], [MX - 165, MY - 10]);
    line(fa, 'ink', 5, 4.15, 4.45, 'B'); line(head(fa, 24), 'ink', 5, 4.45, 4.55, 'B');
    // little example cards streaming along the arrow
    if (t > 4.5 && t < 6.2) {
      for (let k = 0; k < 9; k++) {
        const u = ((t - 4.5) * 0.9 + k / 9) % 1, ix = Math.min(fa.length - 1, Math.floor(u * fa.length));
        const [px, py] = fa[ix], sz = 26 * (1 - 0.5 * u);
        if (t - 4.5 < k / 9 / 0.9) continue;
```

```
        items.push({ k: 'line', pts: rrect(px - sz / 2, py - sz / 2 - 18, sz, sz, 4), role: k % 2 ? 'acc1' : 'acc3', w: 2.5, a: -1, b: 0, g: 'F', inst: true });
      }
    }
    // counter (numerals redrawn on twos)
    if (t > 4.5) {
      const u = win(Math.floor(t * 12) / 12, 4.55, 5.25), n = u >= 1 ? 1e6 : (Math.pow(10, u * 6) - 1) / (1e6 - 1) * 1e6;
      items.push({ k: 'text', s: fmt(n), x: cx0 + 150, y: cy0 + 425, size: 76, font: 'num', role: 'ink', a: -1, b: 0, align: 'center', g: 'B', inst: true, live: true });
      text('examples', cx0 + 150, cy0 + 472, 36, 'label', 'ink', 4.55, 4.8, { align: 'center', g: 'B' });
    }
    if (L.model2At) L.model(MX, MY, mAdj, (it) => { it.g = 'M2'; if (!it.inst) { it.a += L.model2At - 2.5; it.b += L.model2At - 2.5; } items.push(it); }, { line: (p, r, w, a, b) => line(p, r, w, a + L.model2At - 2.5, b + L.model2At - 2.5, 'M2'), text: (s2, x, y, sz, f, r, a, b, o = {}) => text(s2, x, y, sz, f, r, a + L.model2At - 2.5, b + L.model2At - 2.5, { ...o, g: 'M2' }), dot }, t - (L.model2At - 2.5));
    // ---- 3 GUESS -> CHECK -> ADJUST ----
    const qx = MX, qy = MY - 300;
    line(rrect(qx - 62, qy - 62, 124, 124, 12), 'acc2', 4, 6.4, 6.55, 'C');
    cat(qx, qy - 4, 100).forEach((p) => line(p, 'ink', 3.2, 6.5, 6.7, 'C'));
    eyes(qx, qy - 4, 100).forEach(([ex, ey]) => dot(ex, ey, 3, 'ink', 6.7, 'C'));
    text('new!', qx + 72, qy - 30, 30, 'label', 'acc2', 6.65, 6.8, { g: 'C' });
    const dn = [[qx, qy + 70], [qx + 2, MY - 170]];
    line(dn, 'ink', 5, 6.8, 6.95, 'C'); line(head(dn, 18), 'ink', 5, 6.95, 7.02, 'C');
    const gA = [[MX + 175, MY - 10], [MX + 300, MY - 14], [MX + 400, MY - 8]];
    line(gA, 'ink', 5, 7.02, 7.2, 'C'); line(head(gA, 20), 'ink', 5, 7.2, 7.26, 'C');
    text('Guess', MX + 290, MY - 40, 38, 'label', 'ink', 7.15, 7.4, { align: 'center', g: 'C' });
    const BX = MX + 540, BY = MY - 10;
    line(rrect(BX - 120, BY - 55, 240, 110, 40), 'ink', 5, 7.3, 7.5, 'C');
    line([[BX - 70, BY + 52], [BX - 96, BY + 92], [BX - 40, BY + 54]], 'ink', 5, 7.48, 7.55, 'C');
    text('dog?', BX, BY + 18, 52, 'title', 'ink', 7.52, 7.75, { align: 'center', g: 'C' });
    text('Check', BX, BY - 82, 38, 'label', 'acc2', 7.85, 8.05, { align: 'center', g: 'C' });
    line([[BX + 140, BY - 40], [BX + 200, BY + 30]], 'acc2', 8, 8.0, 8.08, 'C');
    line([[BX + 200, BY - 40], [BX + 140, BY + 30]], 'acc2', 8, 8.08, 8.16, 'C');
    line(circ(BX + 4, BY + 2, 150, 82, -2.2, 4.4), 'acc2', 5, 8.15, 8.45, 'C'); // emphasis ring
    const back = cubic([BX - 60, BY + 70], [BX - 80, BY + 290], [MX + 290, MY + 300], [MX + 155, MY + 190]);
    line(back, 'acc2', 6, 8.45, 8.95, 'C'); line(head(back, 22), 'acc2', 6, 8.95, 9.02, 'C');
    text('Adjust', (BX + MX) / 2 - 10, MY + 315, 40, 'label', 'acc2', 8.8, 9.05, { align: 'center', g: 'C' });
    if (t > 8.95 && t < 9.4) { // error signal travelling back
      const u = win(t, 8.95, 9.35), p = back[Math.min(back.length - 1, Math.floor((1 - u) * (back.length - 1)))];
      items.push({ k: 'dot', x: p[0], y: p[1], r: 13, role: 'acc2', a: -1, b: 0, g: 'C', inst: true });
    }
    line([[BX - 66, BY + 4], [BX - 20, BY - 6], [BX + 24, BY + 6], [BX + 66, BY - 4]], 'ink', 5, 9.5, 9.62, 'C'); // strike
```

```
    text('cat', BX + 30, BY + 118, 52, 'title', 'acc3', 9.62, 9.85, { align: 'center', g: 'C' });
    line([[BX + 92, BY + 96], [BX + 112, BY + 122], [BX + 156, BY + 66]], 'acc3', 8, 9.85,
10.05, 'C');
    if (L.bridge) { // a connector the hand draws while the camera travels to the next part
      const B = L.bridge;
      line(B.pts, 'acc1', 5, B.a, B.b, 'D'); line(head(B.pts, 22), 'acc1', 5, B.b, B.b + 0.06, 'D');
      text(B.label, B.lx, B.ly, 40, 'label', 'acc1', B.b + 0.06, B.b + 0.3, { align: 'center', rot: B.rot
|| 0, g: 'D' });
    }
    // ---- 4 PROGRESS ----
    const D = L.D;
    const ox = D[0] - 380, oy = D[1] + 150, cw = 560, ch = 330;
    line([[ox, oy - ch], [ox, oy], [ox + cw, oy]], 'ink', 5, L.pT, L.pT + 0.35, 'D');
    line(head([[ox, oy], [ox, oy - ch]], 16), 'ink', 5, L.pT + 0.35, L.pT + 0.4, 'D');
    line(head([[ox, oy], [ox + cw, oy]], 16), 'ink', 5, L.pT + 0.4, L.pT + 0.45, 'D');
    text('training rounds', ox + cw / 2, oy + 48, 34, 'label', 'ink', L.pT + 0.3, L.pT + 0.55, {
align: 'center', g: 'D' });
    const cpts = [];
    for (let i = 0; i <= 40; i++) {
      const u = i / 40, acc = 61 + 36 * (1 - Math.pow(1 - u, 2.2)) + (i % 3 === 1 ? -2.5 : i % 3
=== 2 ? 1.5 : 0) * (1 - u);
      cpts.push([ox + 20 + u * (cw - 50), oy - (acc - 50) / 50 * ch]);
    }
    line(cpts, 'acc1', 6, L.pT + 0.5, L.pT + 1.7, 'D');
    text('accuracy', D[0] + 330, D[1] - 120, 40, 'label', 'ink', L.pT + 0.45, L.pT + 0.7, { align:
'center', g: 'D' });
    if (t > L.pT + 0.55) {
      const u = win(Math.floor(t * 12) / 12, L.pT + 0.6, L.pT + 1.7), v = 61 + 36 * (1 -
Math.pow(1 - u, 2.2));
      items.push({ k: 'text', s: Math.round(v) + '%', x: D[0] + 330, y: D[1] + 10, size: 110, font:
'num', role: u >= 1 ? 'acc3' : 'ink', a: -1, b: 0, align: 'center', g: 'D', inst: true, live: true });
    }
    line(circ(D[0] + 335, D[1] - 25, 140, 90, -2.4, 4.2), 'acc2', 6, L.pT + 1.75, L.pT + 2.05, 'D');
    // ---- 5 TAKEAWAY ----
    text('More examples → better guesses', D[0], D[1] + 290, 62, 'title', 'ink', L.kT, L.kT +
0.75, { align: 'center', rot: -0.01, g: 'E' });
    line([[D[0] - 450, D[1] + 316], [D[0] + 455, D[1] + 308]], 'acc1', 6, L.kT + 0.75, L.kT + 0.95,
'E');
    return items;
  };
  // the 'tip' of whatever is currently being drawn (for the hand/tool)
  const tipOf = (items, t, measure) => {
    let tip = null;
    for (const it of items) {
      if (it.inst || it.k === 'dot') continue;
      const p = win(t, it.a, it.b);
      if (p > 0 && p < 1) {
```

```
        if (it.k === 'line') { const P = it.pts; tip = P[Math.min(P.length - 1, Math.floor(p *
(P.length - 1)))]; }
        else tip = measure ? measure(it, p) : [it.x, it.y];
       }
      }
      return tip;
    };
    return { clamp, win, eio, camAt, circ, quad, cubic, rrect, head, cat, dog, story, tipOf, fmt };
  })();
  // <<<XE-ENGINE
  registerStyle({
    id: 'blueprint',
    name: 'Engineering Blueprint',
    category: 'Hand-Drawn',
    description: 'A drafted cyanotype: white line-work bleeding slightly into mottled, sun-faded
Prussian-blue paper, orthographic front/side views with a hatched section, dimension chains,
a zoned border, title block and worn fold creases.',
    motion: 'Construction and projection lines sweep across first, object lines ink in view by
view, then dimensions and lettering pop in as the camera slowly pans along the sheet.',
    fonts: ['700 26px "Space Mono"', '400 13px "IBM Plex Mono"', '600 15px "IBM Plex
Mono"', '700 15px "Space Mono"'],
    frameT: 5.4,
    duration: 5.5,
    readsPixels: true,
    draw(ctx, W, H_, t) {
      const nz = H.makeNoise(88);

      // ---------- paper: mottled, sun-faded prussian blue (half-res) ----------
      {
        const w2 = W / 2, h2 = H_ / 2, pl = H.layer(w2, h2), pc = pl.getContext('2d');
        const id = pc.createImageData(w2, h2), d = id.data;
        const base = [22, 58, 112];
        for (let y = 0; y < h2; y++) for (let x = 0; x < w2; x++) {
          const m = nz.fbm2(x * 0.008, y * 0.008, 4) * 0.2 + nz.fbm2(x * 0.05 + 10, y * 0.05, 2) *
0.06 + Math.max(0, nz.fbm2(x * 0.004 + 40, y * 0.004, 3) - 0.15) * 0.5;
          const fade = Math.max(0, 1 - Math.hypot(x - w2 * 0.2, y - h2 * 0.15) / (w2 * 0.9)) * 0.16;
// sun-faded corner
          const ex = Math.min(x, w2 - x) / w2, ey = Math.min(y, h2 - y) / h2;
          const edge = (1 - H.smooth(0, 0.12, Math.min(ex * 1.8, ey))) * 0.3;
          const k = 1 + m + fade - edge;
          const i = (y * w2 + x) * 4;
          d[i] = base[0] * k + fade * 40; d[i + 1] = base[1] * k + fade * 30; d[i + 2] = base[2] * k +
fade * 25; d[i + 3] = 255;
        }
        pc.putImageData(id, 0, 0);
        ctx.imageSmoothingEnabled = true; ctx.drawImage(pl, 0, 0, W, H_);
        // speckle flaws
        const rr = H.rng(4);
```

```
      for (let i = 0; i < 1600; i++) {
        ctx.fillStyle = rr.chance(0.7) ? `rgba(160,190,230,${rr.range(0.05, 0.25)})` :
`rgba(5,20,50,${rr.range(0.1, 0.3)})`;
        ctx.fillRect(rr.range(0, W), rr.range(0, H_), rr.range(0.6, 1.8), rr.range(0.6, 1.8));
      }
    }

    // ---------- line layer ----------
    const LL = H.layer(W, H_), lc = LL.getContext('2d');
    let c = lc;
    const ink = (k) => { k.strokeStyle = '#eef4ff'; k.fillStyle = '#eef4ff'; k.lineCap = 'round';
k.lineJoin = 'round'; };
    ink(c);
    const T = (a, b) => H.clamp((t - a) / (b - a));
    // each group "develops" onto the print behind a soft wipe
    const group = (t0, t1, dir, x0, y0, x1, y1, fn) => {
      const p = T(t0, t1); if (p <= 0) return;
      const GLy = H.layer(W, H_); c = GLy.getContext('2d'); ink(c);
      fn();
      if (p < 1) {
        const e = H.ease.inOut(p), soft = 60;
        const g = dir === 'x' ? c.createLinearGradient(x0 - soft + (x1 - x0 + 2 * soft) * e - soft,
0, x0 - soft + (x1 - x0 + 2 * soft) * e, 0)
          : c.createLinearGradient(0, y0 - soft + (y1 - y0 + 2 * soft) * e - soft, 0, y0 - soft + (y1 -
y0 + 2 * soft) * e);
        g.addColorStop(0, 'rgba(0,0,0,1)'); g.addColorStop(1, 'rgba(0,0,0,0)');
        c.save(); c.globalCompositeOperation = 'destination-in'; c.fillStyle = g; c.fillRect(0, 0,
W, H_); c.restore();
      }
      lc.drawImage(GLy, 0, 0); c = lc;
    };
    let POP = 2.45;
    const pop = () => { const tp = POP; POP += 0.085; return t >= tp; };
    const lw = (w) => { c.lineWidth = w; };
    const dash = (d) => c.setLineDash(d || []);
    const ln = (x0, y0, x1, y1, w = 1, d) => { lw(w); dash(d); c.beginPath(); c.moveTo(x0, y0);
c.lineTo(x1, y1); c.stroke(); dash(); };
    const rr_ = (x, y, w, h, r, lwid = 2.2, d) => { lw(lwid); dash(d); c.beginPath();
c.roundRect(x, y, w, h, r); c.stroke(); dash(); };
    const circ = (x, y, r, lwid = 2.2, d) => { lw(lwid); dash(d); c.beginPath(); c.arc(x, y, r, 0, 7);
c.stroke(); dash(); };
    const CENTER = [18, 4, 3, 4], HIDDEN = [7, 4];
    const arrow = (x, y, ang, s = 11) => {
      c.save(); c.translate(x, y); c.rotate(ang);
      c.beginPath(); c.moveTo(0, 0); c.lineTo(-s, -3); c.lineTo(-s, 3); c.closePath(); c.fill();
c.restore();
    };
    const label = (str, x, y, font = '400 13px "IBM Plex Mono"', align = 'center', rot = 0) => {
```

```
      c.save(); c.font = font; c.textAlign = align; c.textBaseline = 'middle'; c.translate(x, y);
c.rotate(rot); c.fillText(str, 0, 0); c.restore();
    };
    // horizontal dimension: object span x0..x1, extension from yObj to yDim
    const dimH = (x0, x1, yObj, yDim, txt) => {
      if (!pop()) return;
      const s = Math.sign(yDim - yObj) || 1;
      ln(x0, yObj + s * 4, x0, yDim + s * 6, 0.9); ln(x1, yObj + s * 4, x1, yDim + s * 6, 0.9);
      ln(x0, yDim, x1, yDim, 0.9); arrow(x0, yDim, Math.PI); arrow(x1, yDim, 0);
      // gap behind text
      c.save(); c.font = '400 13px "IBM Plex Mono"'; const tw = c.measureText(txt).width;
c.restore();
      c.save(); c.globalCompositeOperation = 'destination-out'; c.fillRect((x0 + x1) / 2 - tw / 2 -
4, yDim - 8, tw + 8, 16); c.restore();
      label(txt, (x0 + x1) / 2, yDim);
    };
    const dimV = (y0, y1, xObj, xDim, txt) => {
      if (!pop()) return;
      const s = Math.sign(xDim - xObj) || 1;
      ln(xObj + s * 4, y0, xDim + s * 6, y0, 0.9); ln(xObj + s * 4, y1, xDim + s * 6, y1, 0.9);
      ln(xDim, y0, xDim, y1, 0.9); arrow(xDim, y0, -Math.PI / 2); arrow(xDim, y1, Math.PI / 2);
      c.save(); c.font = '400 13px "IBM Plex Mono"'; const tw = c.measureText(txt).width;
c.restore();
      c.save(); c.globalCompositeOperation = 'destination-out'; c.fillRect(xDim - 8, (y0 + y1) /
2 - tw / 2 - 4, 16, tw + 8); c.restore();
      label(txt, xDim, (y0 + y1) / 2, undefined, 'center', -Math.PI / 2);
    };
    const leader = (x0, y0, x1, y1, txt, align = 'left') => {
      if (!pop()) return;
      ln(x0, y0, x1, y1, 0.9); ln(x1, y1, x1 + (align === 'left' ? 30 : -30), y1, 0.9);
      c.beginPath(); c.arc(x0, y0, 2.5, 0, 7); c.fill();
      label(txt, x1 + (align === 'left' ? 36 : -36), y1, '400 13px "IBM Plex Mono"', align);
    };

    // ---------- sheet border with zones ----------
    group(0.0, 0.5, 'x', 0, 0, W, H_, () => {
    lw(2.4); c.strokeRect(30, 30, W - 60, H_ - 60);
    lw(1); c.strokeRect(20, 20, W - 40, H_ - 40);
    for (let i = 1; i < 6; i++) { const x = 30 + (W - 60) * i / 6; ln(x, 20, x, 30, 1); ln(x, H_ - 30, x,
H_ - 20, 1); }
    for (let i = 1; i < 4; i++) { const y = 30 + (H_ - 60) * i / 4; ln(20, y, 30, y, 1); ln(W - 30, y, W -
20, y, 1); }
    for (let i = 0; i < 6; i++) { const x = 30 + (W - 60) * (i + 0.5) / 6; label(String(6 - i), x, 25.5,
'400 9px "IBM Plex Mono"'); label(String(6 - i), x, H_ - 25, '400 9px "IBM Plex Mono"'); }
    ['D', 'C', 'B', 'A'].forEach((L, i) => { const y = 30 + (H_ - 60) * (i + 0.5) / 4; label(L, 25, y,
'400 9px "IBM Plex Mono"'); label(L, W - 25, y, '400 9px "IBM Plex Mono"'); });

    });
```

```
    // ---------- FRONT VIEW ----------
    const fx = 270, GY = 560;
    group(0.2, 1.3, 'y', 0, 90, W, 640, () => {
    ln(fx, 88, fx, 584, 0.8, CENTER);
    // antenna + head
    ln(fx, 140, fx, 112, 1.8); circ(fx, 105, 7, 1.8);
    rr_(fx - 62, 140, 124, 86, 14);
    rr_(fx - 48, 158, 96, 42, 10, 1.6);
    circ(fx - 22, 179, 9, 1.6); circ(fx + 22, 179, 9, 1.6);
    rr_(fx - 72, 164, 10, 36, 3, 1.6); rr_(fx + 62, 164, 10, 36, 3, 1.6);
    // neck + torso
    rr_(fx - 18, 226, 36, 18, 2, 1.8);
    rr_(fx - 86, 244, 172, 166, 18);
    rr_(fx - 74, 256, 148, 142, 12, 1, [3, 3]);
    circ(fx, 304, 30, 2); circ(fx, 304, 18, 1.4);
    for (let k = 0; k < 6; k++) { const a = k * Math.PI / 3; ln(fx + Math.cos(a) * 19, 304 +
Math.sin(a) * 19, fx + Math.cos(a) * 29, 304 + Math.sin(a) * 29, 1); }
    rr_(fx - 46, 346, 92, 48, 4, 1.3, HIDDEN); // battery (hidden)
    // shoulders + arms
    [-1, 1].forEach((s) => {
      const ax = fx + s * 104;
      circ(ax, 266, 20, 2); circ(ax, 266, 6, 1.2);
      rr_(ax - 13, 284, 26, 48, 6, 2); circ(ax, 340, 11, 1.8);
      rr_(ax - 12, 350, 24, 46, 6, 2);
      c.beginPath(); lw(2); c.moveTo(ax - 10, 396); c.lineTo(ax - 14, 418); c.lineTo(ax - 4,
426); c.moveTo(ax + 10, 396); c.lineTo(ax + 14, 418); c.lineTo(ax + 4, 426); c.stroke();
    });
    // waist skirt + base + tracks
    c.beginPath(); lw(2.2); c.moveTo(fx - 70, 410); c.lineTo(fx - 96, 470); c.lineTo(fx + 96,
470); c.lineTo(fx + 70, 410); c.stroke();
    for (let k = 0; k < 4; k++) ln(fx - 52 + k * 2, 426 + k * 11, fx + 52 - k * 2, 426 + k * 11, 1.2);
    rr_(fx - 100, 470, 200, 36, 6);
    rr_(fx - 116, 500, 46, 60, 8); rr_(fx + 70, 500, 46, 60, 8);
    for (let k = 0; k < 5; k++) { ln(fx - 116, 510 + k * 10, fx - 70, 510 + k * 10, 0.8); ln(fx + 70,
510 + k * 10, fx + 116, 510 + k * 10, 0.8); }
    // ground line
    ln(110, GY, 760, GY, 1.2);
    for (let x = 116; x < 760; x += 12) ln(x, GY + 1, x - 8, GY + 9, 0.7);
    // section cut A-A
    ln(fx - 140, 304, fx + 140, 304, 1.4, [22, 5, 4, 5]);
    lw(3); c.beginPath(); c.moveTo(fx - 140, 304); c.lineTo(fx - 120, 304); c.moveTo(fx + 120,
304); c.lineTo(fx + 140, 304); c.stroke();
    [-1, 1].forEach((s) => { ln(fx + s * 140, 304, fx + s * 140, 282, 1.4); arrow(fx + s * 140,
280, -Math.PI / 2, 12); label('A', fx + s * 140 + s * 14, 278, '700 15px "Space Mono"'); });
    // dims
    dimV(98, GY, fx - 116, 130, '1450');
    dimV(140, 226, fx + 62, 362, '320');
    dimH(fx - 116, fx + 116, GY, 592, '620');
```

```
    label('FRONT VIEW', fx, 626, '600 15px "IBM Plex Mono"');
    ln(fx - 52, 636, fx + 52, 636, 1);
    leader(fx + 22, 179, fx + 96, 128, 'LiDAR ARRAY');

    });
    // ---------- projection lines ----------
    group(0.95, 1.35, 'x', 400, 0, 520, H_, () => {
    c.save(); c.globalAlpha = 0.35;
    [105, 140, 226, 244, 410, 470, 500].forEach((y) => ln(400, y, 520, y, 0.7, [2, 5]));
    c.restore();

    });
    // ---------- SIDE VIEW ----------
    const sx = 620;
    group(1.1, 2.15, 'y', 0, 90, W, 640, () => {
    ln(sx, 88, sx, 584, 0.8, CENTER);
    ln(sx - 10, 140, sx - 10, 112, 1.8); circ(sx - 10, 105, 7, 1.8);
    rr_(sx - 50, 140, 100, 86, 14);
    rr_(sx + 50, 158, 14, 42, 6, 1.8);
    ln(sx + 20, 150, sx + 20, 216, 1, HIDDEN);
    rr_(sx - 16, 226, 32, 18, 2, 1.8);
    rr_(sx - 60, 244, 120, 166, 18);
    c.beginPath(); lw(2); c.arc(sx + 60, 304, 30, -Math.PI / 2, Math.PI / 2); c.stroke();
    ln(sx + 60, 274, sx + 60, 334, 1.2);
    rr_(sx - 40, 346, 80, 48, 4, 1.3, HIDDEN);
    // the arm sits in front of the torso: knock out what it hides
    c.save(); c.globalCompositeOperation = 'destination-out'; c.beginPath();
    c.arc(sx, 266, 21.5, 0, 7); c.roundRect(sx - 14.5, 282, 29, 51, 7); c.arc(sx, 340, 12.5, 0,
7); c.roundRect(sx - 13.5, 349, 27, 49, 7); c.fill(); c.restore();
    circ(sx, 266, 20, 2); circ(sx, 266, 6, 1.2);
    rr_(sx - 13, 284, 26, 48, 6, 2); circ(sx, 340, 11, 1.8); rr_(sx - 12, 350, 24, 46, 6, 2);
    c.beginPath(); lw(2); c.moveTo(sx + 10, 396); c.lineTo(sx + 26, 414); c.lineTo(sx + 18,
426); c.moveTo(sx - 6, 396); c.lineTo(sx + 2, 420); c.lineTo(sx + 12, 428); c.stroke();
    c.beginPath(); lw(2.2); c.moveTo(sx - 48, 410); c.lineTo(sx - 70, 470); c.lineTo(sx + 70,
470); c.lineTo(sx + 48, 410); c.stroke();
    rr_(sx - 82, 470, 164, 32, 6);
    rr_(sx - 100, 502, 200, 58, 29);
    [-66, 0, 66].forEach((o) => { circ(sx + o, 531, 22, 1.8); circ(sx + o, 531, 7, 1.2); ln(sx + o -
4, 531, sx + o + 4, 531, 0.8); ln(sx + o, 527, sx + o, 535, 0.8); });
    dimH(sx - 100, sx + 100, GY, 592, '480');
    dimV(244, 410, sx - 60, sx - 112, '410');
    label('SIDE VIEW', sx, 626, '600 15px "IBM Plex Mono"');
    ln(sx - 46, 636, sx + 46, 636, 1);
    leader(sx + 60, 266, sx + 110, 228, 'SERVO JOINT ×4');

    });
    // ---------- SECTION A-A ----------
    group(1.85, 2.7, 'x', 900, 0, 1230, H_, () => {
```

```
    {
      const qx = 1050, qy = 290, w = 248, h = 176, th = 12;
      ln(qx - w / 2 - 30, qy, qx + w / 2 + 30, qy, 0.8, CENTER); ln(qx, qy - h / 2 - 30, qx, qy + h / 2 + 30, 0.8, CENTER);
      // hatched shell
      c.save();
      c.beginPath(); c.roundRect(qx - w / 2, qy - h / 2, w, h, 26); c.roundRect(qx - w / 2 + th, qy - h / 2 + th, w - 2 * th, h - 2 * th, 16); c.clip('evenodd');
      for (let o = -400; o < 400; o += 6) ln(qx + o, qy - 120, qx + o + 240, qy + 120, 0.9);
      c.restore();
      rr_(qx - w / 2, qy - h / 2, w, h, 26); rr_(qx - w / 2 + th, qy - h / 2 + th, w - 2 * th, h - 2 * th, 16, 1.6);
      // power core: solid, cross-hatched
      c.save(); c.beginPath(); c.arc(qx, qy, 44, 0, 7); c.arc(qx, qy, 18, 0, 7); c.clip('evenodd');
      for (let o = -120; o < 120; o += 7) { ln(qx + o, qy - 60, qx + o + 120, qy + 60, 0.9); ln(qx + o, qy + 60, qx + o + 120, qy - 60, 0.9); }
      c.restore();
      circ(qx, qy, 44, 2); circ(qx, qy, 18, 1.6);
      // bosses + cable bundle
      [[-1, -1], [1, -1], [-1, 1], [1, 1]].forEach(([a, b]) => { circ(qx + a * (w / 2 - 30), qy + b * (h / 2 - 28), 8, 1.4); circ(qx + a * (w / 2 - 30), qy + b * (h / 2 - 28), 3, 1); });
      [[70, -20], [80, -6], [72, 8], [86, 10]].forEach(([dx, dy]) => circ(qx + dx, qy + dy, 6, 1.2));
      ln(qx + 44, qy, qx + 64, qy - 6, 1.2, HIDDEN);
      dimH(qx - w / 2, qx + w / 2, qy + h / 2, qy + h / 2 + 34, '520');
      dimV(qy - h / 2, qy + h / 2, qx + w / 2, qx + w / 2 + 34, '360');
      leader(qx - 30, qy - 30, qx - 90, qy - 128, 'POWER CORE', 'right');
      leader(qx - w / 2 + 6, qy + 30, qx - w / 2 - 10, qy + 120, 'SHELL t=12', 'right');
      label('SECTION A-A', qx, qy + h / 2 + 76, '600 15px "IBM Plex Mono"');
      ln(qx - 54, qy + h / 2 + 86, qx + 54, qy + h / 2 + 86, 1);
      label('SCALE 1:5', qx, qy + h / 2 + 102, '400 12px "IBM Plex Mono"');
    }

    });
    // ---------- notes ----------
    group(3.35, 3.75, 'x', 50, 0, 420, H_, () => {
    label('NOTES: 1. ALL DIMS IN MM   2. BREAK SHARP EDGES', 52, 668, '400 12px "IBM Plex Mono"', 'left');

    });
    // ---------- title block ----------
    group(3.0, 3.6, 'x', 868, 0, W - 30, H_, () => {
    {
      const x0 = 868, y0 = 560, x1 = W - 30, y1 = H_ - 30;
      lw(2.2); c.strokeRect(x0, y0, x1 - x0, y1 - y0);
      ln(x0, y0 + 56, x1, y0 + 56, 1.2); ln(x0, y0 + 92, x1, y0 + 92, 1.2);
      const cw = (x1 - x0) / 3;
      ln(x0 + cw, y0 + 56, x0 + cw, y1, 1.2); ln(x0 + 2 * cw, y0 + 56, x0 + 2 * cw, y1, 1.2);
      label('SERVICE ROBOT MK-II', x0 + 14, y0 + 24, '700 26px "Space Mono"', 'left');
```

```
      label('GENERAL ARRANGEMENT', x0 + 14, y0 + 46, '400 12px "IBM Plex Mono"',
'left');
      const cell = (i, row, k, v) => {
        label(k, x0 + cw * i + 8, y0 + (row ? 102 : 66), '400 9px "IBM Plex Mono"', 'left');
        label(v, x0 + cw * i + 8, y0 + (row ? 118 : 82), '600 15px "IBM Plex Mono"', 'left');
      };
      cell(0, 0, 'DWG NO.', 'SR-2042'); cell(1, 0, 'SCALE', '1:10'); cell(2, 0, 'SHEET', '1 OF 3');
      cell(0, 1, 'DRAWN', 'J.K.'); cell(1, 1, 'DATE', '09.10.26'); cell(2, 1, 'REV', 'B');
    }

    });
    // ---------- cyanotype line character: uneven exposure + bleed ----------
    {
      const id = lc.getImageData(0, 0, W, H_), d = id.data;
      for (let i = 3, p = 0; i < d.length; i += 4, p++) {
        if (!d[i]) continue;
        const x = p % W, y = (p / W) | 0;
        const k = 0.72 + 0.28 * (0.5 + 0.5 * nz.n2(x * 0.03, y * 0.03)) - (nz.n2(x * 0.5, y * 0.5) >
0.55 ? 0.3 : 0);
        d[i] = d[i] * k;
      }
      lc.putImageData(id, 0, 0);
    }
    ctx.save(); ctx.filter = 'blur(2.5px)'; ctx.globalAlpha = 0.45; ctx.drawImage(LL, 0, 0);
ctx.restore();
    ctx.save(); ctx.filter = 'blur(0.6px)'; ctx.globalAlpha = 0.95; ctx.drawImage(LL, 0, 0);
ctx.restore();

    // fine grid (printed underneath conceptually, but faint so draw after)
    ctx.save();
    for (let x = 30; x < W - 30; x += 20) { ctx.fillStyle = (x - 30) % 100 === 0 ?
'rgba(200,220,255,0.10)' : 'rgba(200,220,255,0.045)'; ctx.fillRect(x, 30, 1, H_ - 60); }
    for (let y = 30; y < H_ - 30; y += 20) { ctx.fillStyle = (y - 30) % 100 === 0 ?
'rgba(200,220,255,0.10)' : 'rgba(200,220,255,0.045)'; ctx.fillRect(30, y, W - 60, 1); }
    ctx.restore();

    // ---------- fold creases + wear ----------
    {
      const rr = H.rng(31);
      const fold = (vertical, pos) => {
        const g = vertical ? ctx.createLinearGradient(pos - 40, 0, pos + 40, 0) :
ctx.createLinearGradient(0, pos - 34, 0, pos + 34);
        g.addColorStop(0, 'rgba(0,10,30,0)'); g.addColorStop(0.48, 'rgba(0,10,30,0.22)');
g.addColorStop(0.5, 'rgba(190,210,240,0.22)'); g.addColorStop(0.56,
'rgba(190,210,240,0.06)'); g.addColorStop(1, 'rgba(190,210,240,0)');
        ctx.fillStyle = g;
        if (vertical) ctx.fillRect(pos - 40, 0, 80, H_); else ctx.fillRect(0, pos - 34, W, 68);
        // abraded fibres along the crease
```

```
        for (let i = 0; i < 700; i++) {
          const a = rr.range(0, vertical ? H_ : W), o = rr.gauss() * 1.8;
          ctx.fillStyle = `rgba(200,220,245,${rr.range(0.1, 0.45)})`;
          const s = rr.range(0.6, 2.2);
          if (vertical) ctx.fillRect(pos + o, a, s * 0.7, s); else ctx.fillRect(a, pos + o, s, s * 0.7);
        }
      };
      fold(true, 427); fold(true, 853); fold(false, 362);
      // crossing points wear most
      [[427, 362], [853, 362]].forEach(([x, y]) => {
        const g = ctx.createRadialGradient(x, y, 0, x, y, 26); g.addColorStop(0,
'rgba(200,220,245,0.28)'); g.addColorStop(1, 'rgba(200,220,245,0)');
        ctx.fillStyle = g; ctx.fillRect(x - 26, y - 26, 52, 52);
      });
    }
    // slow camera drift across the sheet, settling on the full drawing
    {
      const e = H.ease.inOut(T(0, 4.6)), k = 1 + 0.05 * (1 - e);
      if (k > 1.0005) {
        const cv = ctx.canvas, snap = H.layer(cv.width, cv.height);
        snap.getContext('2d').drawImage(cv, 0, 0);
        const fx0 = W * (0.3 + 0.2 * e), fy0 = H_ * (0.4 + 0.1 * e);
        ctx.save(); ctx.translate(fx0, fy0); ctx.scale(k, k); ctx.translate(-fx0, -fy0);
ctx.drawImage(snap, 0, 0, W, H_); ctx.restore();
      }
    }
    H.grain(ctx, W, H_, 0.04, 6 + (Math.floor(t * 12) % 4));
    H.vignette(ctx, W, H_, 0.3, '0,8,25');
  },
  // >>>EXPLAINER
  explainerDuration: 15,
  explainerNote: 'One large drafting sheet: a plotter head inks crisp white line-work and
stencilled lettering while the camera tracks across and then down the sheet to the chart
detail; the model is a gear-and-knob mechanism whose dials turn, arrows are drafted
leaders, and numbers tick over in mono lettering inside dimension-style callouts.',
  explainer(ctx, W, H_, t) {
    const nz = H.makeNoise(88);
    const seg = (pts, step = 3) => {
      const out = [];
      for (let i = 0; i < pts.length - 1; i++) {
        const [x0, y0] = pts[i], [x1, y1] = pts[i + 1], n = Math.max(1, Math.ceil(Math.hypot(x1 -
x0, y1 - y0) / step));
        for (let j = 0; j < n; j++) out.push([x0 + (x1 - x0) * j / n, y0 + (y1 - y0) * j / n]);
      }
      out.push(pts[pts.length - 1]); return out;
    };
    const D = [2560, 1520];
    const L = {
```

```
      D, pT: 10.95, kT: 13.05, modelName: 'MODEL  UNIT M-1',
      bridge: { pts: [[2760, 1010], [2760, 1150], [2700, 1210], [2620, 1290]], a: 10.15, b: 10.6,
label: 'REPEAT x1000', lx: 2960, ly: 1250 },
      model: (cx, cy, adj, push, api, tt) => {
        const g = 'M';
        api.line([[cx - 165, cy - 150], [cx + 165, cy - 150], [cx + 165, cy + 175], [cx - 165, cy +
175], [cx - 165, cy - 150]], 'ink', 5, 2.5, 2.8, g);
        api.line([[cx - 150, cy - 135], [cx + 150, cy - 135], [cx + 150, cy + 160], [cx - 150, cy +
160], [cx - 150, cy - 135]], 'ink', 2, 2.75, 2.95, g);
        api.line([[cx - 55, cy - 150], [cx - 85, cy - 195], [cx + 85, cy - 195], [cx + 55, cy - 150]],
'ink', 4, 2.85, 2.98, g);
        const gear = (x, y, r, n, rot) => { const o = []; for (let i = 0; i <= n * 4; i++) { const k = i %
4, b = Math.floor(i / 4) * (Math.PI * 2 / n) + rot; const a = b + [0, 0.1, 0.4, 0.5][k] * (Math.PI * 2
/ n); const rr = k === 1 || k === 2 ? r : r * 0.82; o.push([x + Math.cos(a) * rr, y + Math.sin(a) *
rr]); } return o; };
        const spin = adj * 1.4;
        const gears = [[cx - 60, cy - 40, 62, 12, spin], [cx + 58, cy - 62, 44, 9, -spin * 1.38 +
0.2]];
        gears.forEach(([x, y, r, n, rot], i) => {
          if (adj > 0) push({ k: 'line', pts: gear(x, y, r, n, rot), role: adj < 1 ? 'acc2' : 'ink', w: 3.5, a:
-1, b: 0, inst: true, g });
          else api.line(gear(x, y, r, n, rot), 'ink', 3.5, 2.95 + i * 0.12, 3.12 + i * 0.12, g);
          api.line(XE.circ(x, y, r * 0.3, r * 0.3), 'ink', 2.5, 3.1 + i * 0.1, 3.16 + i * 0.1, g);
          api.line([[x - r - 12, y], [x + r + 12, y]], 'cl', 1.2, 3.0, 3.1, g);
          api.line([[x, y - r - 12], [x, y + r + 12]], 'cl', 1.2, 3.0, 3.1, g);
        });
        [-1, 0, 1].forEach((k, i) => {
          const x = cx + k * 90, y = cy + 100;
          api.line(XE.circ(x, y, 26, 26), 'ink', 3.5, 3.15 + i * 0.05, 3.25 + i * 0.05, g);
          for (let m = 0; m < 7; m++) { const a = -Math.PI + m * Math.PI / 6; api.line([[x +
Math.cos(a) * 30, y + Math.sin(a) * 30], [x + Math.cos(a) * 37, y + Math.sin(a) * 37]], 'ink', 1.5,
3.25 + i * 0.05, 3.27 + i * 0.05, g); }
          if (tt > 3.25 + i * 0.05) {
            const a0 = [-2.4, -0.6, -1.9][i], a1 = [-0.7, -2.2, -0.4][i], a = a0 + (a1 - a0) * adj +
Math.sin(adj * Math.PI) * 0.3;
            push({ k: 'line', pts: [[x, y], [x + Math.cos(a) * 22, y + Math.sin(a) * 22]], role: adj > 0
&& adj < 1 ? 'acc2' : 'ink', w: 4, a: -1, b: 0, inst: true, g });
          }
        });
      },
    };
    const items = XE.story(t, L);
    const cam = XE.camAt([[0, 640, 360, 1.0], [2.0, 650, 362, 1.05], [2.8, 1830, 610, 0.95],
[5.95, 1860, 620, 0.97], [6.55, 2580, 640, 0.9], [10.3, 2600, 660, 0.93],
      [11.0, 2560, 1540, 0.92], [12.9, 2570, 1550, 0.95], [13.6, 2680, 1620, 0.84], [15, 2680,
1620, 0.85]], t);
    const z = cam[2];
```

```
    const apply = (c) => { c.translate(W / 2, H_ / 2); c.scale(z, z); c.translate(-cam[0],
-cam[1]); };
    const toS = ([x, y]) => [(x - cam[0]) * z + W / 2, (y - cam[1]) * z + H_ / 2];
    const vx0 = cam[0] - W / 2 / z, vx1 = cam[0] + W / 2 / z, vy0 = cam[1] - H_ / 2 / z, vy1 =
cam[1] + H_ / 2 / z;

    // ---------- the sheet (mottled prussian blue, built once; 2 world px per texel so the mottle
stays crisp) ----------
    // world sheet: outer border 232..3418 x 42..2068; the paper's own edge sits a little
outside it and darkens like the still's
    const PX0 = 172, PY0 = -18, PX1 = 3478, PY1 = 2128;
    if (!XC.sheet) {
      const sw = 2000, sh = 1300, sl = H.layer(sw, sh), sc = sl.getContext('2d'), id =
sc.createImageData(sw, sh), d = id.data, base = [22, 58, 112];
      // the broad terms are smooth, so they are sampled on coarser grids and interpolated;
only the fine mottle is per texel
      const grid = (step, fn) => { const gw = Math.ceil(sw / step) + 2, gh = Math.ceil(sh / step)
+ 2, G = new Float32Array(gw * gh);
        for (let j = 0; j < gh; j++) for (let i = 0; i < gw; i++) G[j * gw + i] = fn(i * step, j * step);
        return (x, y) => { const fx = x / step, fy = y / step, i = fx | 0, j = fy | 0, u = fx - i, v = fy - j,
o = j * gw + i;
          return (G[o] * (1 - u) + G[o + 1] * u) * (1 - v) + (G[o + gw] * (1 - u) + G[o + gw + 1] * u)
* v; }; };
      const gA = grid(2, (x, y) => nz.fbm2(x * 0.006, y * 0.006, 4) * 0.24), gB = grid(4, (x, y) =>
Math.max(0, nz.fbm2(x * 0.003 + 40, y * 0.003, 3) - 0.15) * 0.55);
      // fine mottle: cheap 2-octave value noise per texel
      const VN = 256, vl = new Float32Array(VN * VN), vr = H.rng(881); for (let i = 0; i <
vl.length; i++) vl[i] = vr() * 2 - 1;
      const vn = (x, y) => { const ix = Math.floor(x), iy = Math.floor(y), u = x - ix, v = y - iy, su =
u * u * (3 - 2 * u), sv = v * v * (3 - 2 * v);
        const x0 = ix & 255, x1 = (ix + 1) & 255, y0 = (iy & 255) * VN, y1 = ((iy + 1) & 255) *
VN;
        const a = vl[y0 + x0], b = vl[y0 + x1], c2 = vl[y1 + x0], e = vl[y1 + x1]; return (a + (b - a)
* su) * (1 - sv) + (c2 + (e - c2) * su) * sv; };
      for (let y = 0; y < sh; y++) for (let x = 0; x < sw; x++) {
        const m = gA(x, y) + (vn(x * 0.06 + 10, y * 0.06) * 0.65 + vn(x * 0.14, y * 0.14 + 7) *
0.35) * 0.07 + gB(x, y);
        const fade = Math.max(0, 1 - Math.hypot(x - 240, y - 160) / 1400) * 0.16;
        const wx = x * 2 - 300, wy = y * 2 - 300, dE = Math.min(wx - PX0, PX1 - wx, wy - PY0,
PY1 - wy);
        const edge = (1 - H.smooth(0, 230, dE)) * 0.3;
        const k = 1 + m + fade - edge, i = (y * sw + x) * 4;
        d[i] = base[0] * k + fade * 40; d[i + 1] = base[1] * k + fade * 30; d[i + 2] = base[2] * k +
fade * 25; d[i + 3] = 255;
      }
      sc.putImageData(id, 0, 0); XC.sheet = sl;
      // speckle flaws (the still's light/dark flecks), as a 1024px tile anchored to the paper
      const TS = 1024, tl = H.layer(TS, TS), tc = tl.getContext('2d'), rr = H.rng(4);
```

```
      for (let i = 0; i < 1900; i++) {
        tc.fillStyle = rr.chance(0.7) ? `rgba(160,190,230,${rr.range(0.05, 0.25)})` :
`rgba(5,20,50,${rr.range(0.1, 0.3)})`;
        tc.fillRect(rr.range(0, TS - 2), rr.range(0, TS - 2), rr.range(0.6, 1.8), rr.range(0.6, 1.8));
      }
      XC.speck = tl;
      // abraded fibres along the fold creases (the still's fold() scatter), pre-rendered as strips
      const fr = H.rng(31), fib = (vert) => {
        const len = vert ? 2600 : 4000, st = vert ? H.layer(24, len) : H.layer(len, 24), fc =
st.getContext('2d');
        for (let i = 0; i < 2300; i++) {
          const a = fr.range(0, len), o = 12 + fr.gauss() * 1.8, sz = fr.range(0.6, 2.2);
          fc.fillStyle = `rgba(200,220,245,${fr.range(0.1, 0.45)})`;
          if (vert) fc.fillRect(o, a, sz * 0.7, sz); else fc.fillRect(a, o, sz, sz * 0.7);
        }
        return st;
      };
      XC.fibV = [fib(true), fib(true)]; XC.fibH = fib(false);
    }
    ctx.save(); apply(ctx);
    ctx.imageSmoothingEnabled = true; ctx.drawImage(XC.sheet, -300, -300, 4000, 2600);
    ctx.fillStyle = ctx.createPattern(XC.speck, 'repeat'); ctx.fillRect(vx0 - 4, vy0 - 4, vx1 - vx0
+ 8, vy1 - vy0 + 8);
    for (let x = Math.floor(vx0 / 40) * 40; x < vx1; x += 40) { ctx.fillStyle = x % 200 === 0 ?
'rgba(200,220,255,0.10)' : 'rgba(200,220,255,0.045)'; ctx.fillRect(x, vy0, 1 / z, vy1 - vy0); }
    for (let y = Math.floor(vy0 / 40) * 40; y < vy1; y += 40) { ctx.fillStyle = y % 200 === 0 ?
'rgba(200,220,255,0.10)' : 'rgba(200,220,255,0.045)'; ctx.fillRect(vx0, y, vx1 - vx0, 1 / z); }
    ctx.restore();

    // ---------- line work ----------
    const LL = H.layer(W, H_), c = LL.getContext('2d');
    c.save(); apply(c);
    c.strokeStyle = '#eef4ff'; c.fillStyle = '#eef4ff'; c.lineCap = 'round'; c.lineJoin = 'round';
    // sheet border, zone ticks and title block
    c.lineWidth = 4; c.strokeRect(250, 60, 3150, 1990); c.lineWidth = 1.5; c.strokeRect(232,
42, 3186, 2026);
    for (let x = 250; x <= 3400; x += 525) { c.beginPath(); c.moveTo(x, 42); c.lineTo(x, 60);
c.moveTo(x, 2050); c.lineTo(x, 2068); c.stroke(); }
    c.lineWidth = 3; c.strokeRect(3030, 1880, 370, 170); c.lineWidth = 1.5;
    c.beginPath(); c.moveTo(3030, 1945); c.lineTo(3400, 1945); c.moveTo(3150, 1945);
c.lineTo(3150, 2050); c.moveTo(3275, 1945); c.lineTo(3275, 2050); c.stroke();
    c.font = '700 32px "Space Mono"'; c.fillText('HOW AI LEARNS', 3045, 1925);
    c.font = '400 16px "IBM Plex Mono"'; c.fillText('DWG NO.', 3042, 1968);
c.fillText('SCALE', 3162, 1968); c.fillText('SHEET', 3287, 1968);
    c.font = '600 24px "IBM Plex Mono"'; c.fillText('ML-0001', 3042, 2015); c.fillText('NTS',
3162, 2015); c.fillText('1/1', 3287, 2015);
    const font = (it) => (it.font === 'label' ? `600 ${it.size * 0.86}px "IBM Plex Mono"` : `700
${it.size * (it.font === 'num' ? 0.95 : 0.9)}px "Space Mono"`);
```

```
    const measure = (it, p) => {
      c.save(); c.font = font(it); const s = it.s.toUpperCase(), w = c.measureText(s).width, x0 =
it.align === 'center' ? it.x - w / 2 : it.x;
      const w2 = c.measureText(s.slice(0, Math.ceil(s.length * p))).width; c.restore(); return
[x0 + w2, it.y - it.size * 0.3];
    };
    for (const it of items) {
      const p = it.inst ? 1 : XE.win(t, it.a, it.b);
      if (p <= 0) continue;
      const alpha = it.role === 'acc2' ? 1 : it.role === 'cl' ? 0.7 : 0.95;
      c.globalAlpha = alpha;
      if (it.k === 'line') {
        let P = seg(it.pts, 3); if (p < 1) P = P.slice(0, Math.max(2, Math.ceil(P.length * p)));
        c.lineWidth = it.role === 'acc2' ? it.w * 0.75 : Math.max(1.8, it.w * 0.55);
        c.setLineDash(it.role === 'cl' ? [22, 5, 4, 5] : it.role === 'acc2' && it.w <= 6 &&
it.pts.length > 30 ? [16, 7] : []);
        c.beginPath(); P.forEach(([x, y], i) => (i ? c.lineTo(x, y) : c.moveTo(x, y))); c.stroke();
c.setLineDash([]);
        // drafted arrows: fill arrowheads
        if (it.pts.length === 3 && it.w >= 4 && p >= 1 && Math.hypot(it.pts[0][0] - it.pts[2][0],
it.pts[0][1] - it.pts[2][1]) < 40) { c.beginPath(); c.moveTo(...it.pts[0]); c.lineTo(...it.pts[1]);
c.lineTo(...it.pts[2]); c.closePath(); c.fill(); }
      } else if (it.k === 'dot') {
        c.beginPath(); c.arc(it.x, it.y, it.r * 0.8, 0, 7); c.fill();
      } else {
        c.save(); c.font = font(it); const s = it.s.toUpperCase(), w = c.measureText(s).width;
        c.translate(it.align === 'center' ? it.x - w / 2 : it.x, it.y); c.rotate(it.rot * 0.3);
        c.fillText(s.slice(0, Math.ceil(s.length * p)), 0, 0);
        if (it.live) { c.globalAlpha = 0.6; c.lineWidth = 1.5; c.strokeRect(-14, -it.size * 0.8, w +
28, it.size * 1.0); } // counter callout box
        c.restore();
      }
    }
    c.globalAlpha = 1; c.restore();
    // uneven exposure: knock a little out of the lines (anchored to the sheet)
    {
      const id = c.getImageData(0, 0, W, H_), d = id.data, iz = 1 / z;
      for (let i = 3, q = 0; i < d.length; i += 4, q++) {
        if (!d[i]) continue;
        const x = q % W, y = (q / W) | 0, wx = (x - W / 2) * iz + cam[0], wy = (y - H_ / 2) * iz +
cam[1];
        // + the still's pinhole dropouts (unexposed specks in the white line), hashed on 2px
world cells
        const hx = (wx * 0.5) | 0, hy = (wy * 0.5) | 0, hh = Math.imul(hx * 73856093 ^ hy *
19349663, 0x9E3779B1) >>> 0;
        d[i] *= 0.74 + 0.26 * (0.5 + 0.5 * nz.n2(wx * 0.03, wy * 0.03)) - ((hh & 1023) < 70 ? 0.3 :
0);
      }
```

```
      c.putImageData(id, 0, 0);
    }
    ctx.save(); ctx.filter = 'blur(2.5px)'; ctx.globalAlpha = 0.45; ctx.drawImage(LL, 0, 0);
ctx.restore();
    ctx.save(); ctx.filter = 'blur(0.5px)'; ctx.drawImage(LL, 0, 0); ctx.restore();

    // fold creases (on the sheet)
    ctx.save(); apply(ctx);
    [[1480, true], [2760, true], [1180, false]].forEach(([pos, vert]) => {
      const g = vert ? ctx.createLinearGradient(pos - 40, 0, pos + 40, 0) :
ctx.createLinearGradient(0, pos - 34, 0, pos + 34);
      g.addColorStop(0, 'rgba(0,10,30,0)'); g.addColorStop(0.48, 'rgba(0,10,30,0.22)');
g.addColorStop(0.5, 'rgba(190,210,240,0.22)'); g.addColorStop(0.56,
'rgba(190,210,240,0.06)'); g.addColorStop(1, 'rgba(190,210,240,0)');
      ctx.fillStyle = g; if (vert) ctx.fillRect(pos - 40, vy0, 80, vy1 - vy0); else ctx.fillRect(vx0, pos
- 34, vx1 - vx0, 68);
    });
    // abraded fibres along each crease, and the extra wear where the creases cross
    if (vx0 < 1500 && vx1 > 1460) ctx.drawImage(XC.fibV[0], 1480 - 12, -300);
    if (vx0 < 2780 && vx1 > 2740) ctx.drawImage(XC.fibV[1], 2760 - 12, -300);
    if (vy0 < 1200 && vy1 > 1160) ctx.drawImage(XC.fibH, -300, 1180 - 12);
    [[1480, 1180], [2760, 1180]].forEach(([x, y]) => {
      if (x + 70 < vx0 || x - 70 > vx1 || y + 70 < vy0 || y - 70 > vy1) return;
      const g = ctx.createRadialGradient(x, y, 0, x, y, 64); g.addColorStop(0,
'rgba(200,220,245,0.28)'); g.addColorStop(1, 'rgba(200,220,245,0)');
      ctx.fillStyle = g; ctx.fillRect(x - 64, y - 64, 128, 128);
    });
    ctx.restore();

    // plotter head over the current stroke
    const tp = XE.tipOf(items, t, measure);
    if (tp) {
      const [x, y] = toS(tp);
      const gl = ctx.createRadialGradient(x, y, 0, x, y, 26); gl.addColorStop(0,
'rgba(230,245,255,0.8)'); gl.addColorStop(1, 'rgba(230,245,255,0)');
      ctx.fillStyle = gl; ctx.fillRect(x - 26, y - 26, 52, 52);
      ctx.strokeStyle = 'rgba(240,248,255,0.9)'; ctx.lineWidth = 1.2;
      ctx.beginPath(); ctx.arc(x, y, 9, 0, 7); ctx.moveTo(x - 18, y); ctx.lineTo(x - 5, y);
ctx.moveTo(x + 5, y); ctx.lineTo(x + 18, y); ctx.moveTo(x, y - 18); ctx.lineTo(x, y - 5);
ctx.moveTo(x, y + 5); ctx.lineTo(x, y + 18); ctx.stroke();
      ctx.setLineDash([3, 4]); ctx.globalAlpha = 0.35; ctx.beginPath(); ctx.moveTo(x, 0);
ctx.lineTo(x, H_); ctx.moveTo(0, y); ctx.lineTo(W, y); ctx.stroke(); ctx.setLineDash([]);
ctx.globalAlpha = 1;
    }
    H.grain(ctx, W, H_, 0.04, 6 + (Math.floor(t * 12) % 4));
    H.vignette(ctx, W, H_, 0.3, '0,8,25');
  },
  // <<<EXPLAINER
```

```
    // Sound: a pen plotter drafting on a big sheet in a quiet drawing office. Each line is a plot move: the pen solenoid
    // drops (a dull "tuk"), the stepper motors sing a low square-ish whine whose pitch changes from move to move while the
    // technical pen drags faintly, then the pen lifts. Stencil lettering is quick pen-down ticks with tiny motor blips per
    // character. The drafted "REPEAT x1000" leader is a long carriage run that carries the camera down the sheet; the head
    // parks when the drawing is done. Times mirror explainer()/XE.story (pT 10.95, kT 13.05, bridge 10.15-10.6, model
    // timings from explainer()).
    explainerSound(k) {
      const pT = 10.95, kT = 13.05, LV = 0.5; // LV: drawing sits well under the hits
      const rnd = (i) => { const x = Math.sin(i * 12.9898 + 78.233) * 43758.5453; return x - Math.floor(x); };
      // first time XE.eio(win(t, t0, t1)) reaches u (the gears' and dials' eased turn)
      const when = (t0, t1, u) => { for (let i = 0; i <= 400; i++) if (XE.eio(i / 400) >= u - 1e-6) return t0 + (t1 - t0) * i / 400; return t1; };
      const penDown = (t, g = 1, pan = 0, minor = true) => {
        k.noise(t, { dur: 0.02, filter: 'bandpass', f: 1300, q: 1.5, gain: 0.25 * g, a: 0.001, pan, minor });
        k.tone(t, { f: [[0, 180], [0.03, 110]], dur: 0.045, gain: 0.1 * g, pan, minor });
      };
      const penUp = (t, g = 1, pan = 0) => k.noise(t, { dur: 0.012, filter: 'bandpass', f: 1900, q: 2, gain: 0.12 * g, a: 0.001, pan, minor: true });
      // the steppers during a move: a square whine (lowpassed) plus a sub an octave down; f = steps per second
      const motor = (t, d, f, g = 1, pan = 0, minor = true) => {
        k.tone(t, { f, type: 'square', dur: d, gain: 0.03 * g, a: 0.012, hold: d * 0.75, filter: 'lowpass', f2: 950, q: 0.7, pan, minor });
        k.tone(t, { f: f / 2, type: 'triangle', dur: d, gain: 0.05 * g, a: 0.012, hold: d * 0.75, pan, minor });
      };
      // one drafted line from a to b: pen down, move, pen up. The pitch is per-move (seeded from its start time).
      const plot = (a, b, g = 1, pan = 0, minor = true, f = 0) => {
        const d = Math.max(0.03, b - a); g *= LV;
        f = f || 200 + Math.round(rnd(a * 100) * 6) * 25;
        penDown(a, g, pan, minor);
        motor(a + 0.01, d, f, g, pan, minor);
        k.noise(a, { dur: d, filter: 'bandpass', f: 2800, q: 0.9, gain: 0.03 * g, a: 0.02, hold: d * 0.7, pan, minor }); // pen drag
        penUp(b, g, pan);
      };
      // stencil lettering: per character a pen tick and a short motor blip (spaces are pen-up travel);
      // long fast lines tick on every 2nd/3rd character so it stays under ~20 ticks a second
      const letter = (s, a, b, g = 0.9, pan = 0) => {
```

```
      g *= LV; const n = s.length, step = (b - a) / n, stride = Math.max(1, Math.ceil(n / (b - a) /
20));
      for (let i = 0; i < n; i++) {
        if (s[i] === ' ' || i % stride) continue;
        const t = a + step * i, c = s.charCodeAt(i);
        k.noise(t, { dur: 0.012, filter: 'bandpass', f: 1500, q: 1.6, gain: 0.2 * g, a: 0.001, pan,
minor: true });
        motor(t + 0.005, Math.min(0.06, step * stride * 0.75), 220 + (c % 5) * 30, 0.8 * g, pan,
true);
      }
    };

    k.bed('room', 0, 15, { gain: 0.35 });
    k.bed('hum', 0, 15, { gain: 0.25 }); // the plotter powered up, idling

    // 1 HOOK
    letter('HOW DOES AI LEARN?', 0.15, 1.3, 1.0);
    plot(1.3, 1.55, 1.3, 0, false, 300); // underline: one long straight move

    // 2 DATA: the gear-and-dial unit, then six cards
    plot(2.5, 2.8, 1.1, 0.1, false, 250); // outer frame: first mark in the new scene
    plot(2.8, 2.95, 0.7, 0.1); // inner frame
    plot(2.95, 3.19, 0.8, 0.1, true, 175); // the two gears, tooth by tooth (lower, slower
stepping)
    for (let i = 0; i < 6; i++) { const a = 3.0 + i * 0.17; plot(a + 0.02, a + (i < 5 ? 0.15 : 0.21),
0.8, (i % 3 - 1) * 0.15); }
    for (let j = 0; j < 4; j++) penDown(3.0 + j * 0.025, 0.35 * LV, 0.15); // centre-line dashes
    plot(4.15, 4.45, 1.0, 0, true, 275); plot(4.46, 4.55, 0.8); // flow leader + arrowhead
    letter('EXAMPLES', 4.55, 4.8, 0.6);
    // counter digits roll over in the callout (12 fps grid): a small relay click each change
    for (let f = 55; f < 63; f++) { k.play('count', f / 12, { gain: 1.4, f: 2000 + (f - 55) * 100, minor:
true }); k.tone(f / 12, { f: 140, dur: 0.03, gain: 0.04, minor: true }); }
    k.play('tock', 63 / 12, { f: 480, gain: 0.45 }); // 1,000,000 lands

    // 3 GUESS -> CHECK -> ADJUST
    plot(6.4, 6.55, 0.9, 0, true, 250); // frame of the new photo
    plot(6.55, 6.7, 0.8); penDown(6.7, 0.5 * LV); // cat + eyes
    letter('NEW!', 6.65, 6.8, 0.6, 0.2);
    plot(6.8, 6.95, 0.9, 0, true, 225); plot(6.96, 7.02, 0.7); // leader down into the unit + head
    plot(7.02, 7.15, 0.9, 0, true, 275); // guess leader out
    letter('GUESS', 7.15, 7.4, 0.6);
    plot(7.3, 7.55, 0.85, 0, true, 200); // speech bubble + tail
    letter('DOG?', 7.52, 7.75, 1.0);
    letter('CHECK', 7.85, 8.0, 0.6);
    // the wrong guess: two hard cross strokes (pen slams down) + a low "nope"
    plot(8.0, 8.08, 2.0, 0.1, false, 350); plot(8.08, 8.16, 2.0, 0.1, false, 350);
    k.play('error', 8.0, { f: 'F3', gain: 0.55 });
    plot(8.16, 8.45, 1.0, 0, true, 225); // dashed ring
```

```
    plot(8.45, 8.8, 1.0, -0.1, true, 200); // long dashed leader back to the unit
    letter('ADJUST', 8.8, 9.05, 0.6, -0.1);
    // gears and dials turn (eio 8.95..9.5, gear 1.4 rad): a soft detent click every 0.2 rad, slowing as they settle
    for (let j = 1; j <= 7; j++) k.play('tock', when(8.95, 9.5, j / 7), { f: 900, gain: 0.3, pan: -0.15, minor: true });
    plot(9.5, 9.62, 1.0, 0, true, 300); // strike through "DOG?"
    letter('CAT', 9.62, 9.85, 1.0);
    plot(9.85, 10.05, 1.15, 0, false, 250); // check mark
    k.play('success', 9.98, { f: 'A5', gain: 0.6 });

    // bridge: the "REPEAT x1000" leader, a long carriage run while the camera tracks down the sheet
    penDown(10.15, 1.3 * LV, 0.2, false);
    k.tone(10.16, { f: [[0, 180], [0.2, 320], [0.45, 240]], type: 'square', dur: 0.45, gain: 0.03 * LV * 1.4, a: 0.02, hold: 0.36, filter: 'lowpass', f2: 950, q: 0.7, pan: 0.2 });
    k.tone(10.16, { f: [[0, 90], [0.2, 160], [0.45, 120]], type: 'triangle', dur: 0.45, gain: 0.05 * LV * 1.4, a: 0.02, hold: 0.36, pan: 0.2 });
    k.noise(10.15, { dur: 0.45, filter: 'bandpass', f: 2800, q: 0.9, gain: 0.03 * LV * 1.4, a: 0.02, hold: 0.3, pan: 0.2, minor: true });
    plot(10.6, 10.66, 0.8, 0.25);
    letter('REPEAT X1000', 10.66, 10.9, 0.7, 0.3);

    // 4 PROGRESS
    plot(pT, pT + 0.3, 1.0, 0, false, 250); // axes
    letter('TRAINING ROUNDS', pT + 0.3, pT + 0.55, 0.5);
    letter('ACCURACY', pT + 0.45, pT + 0.7, 0.5, 0.3);
    plot(pT + 0.7, pT + 1.7, 0.45, 0, true, 175); // the rising curve: one long slow move
    { // % counter: a relay click whenever the shown number changes (12 fps grid), slowing as it settles.
      // The digits don't change colour, so the hit lands on the frame 97% first shows.
      let last = -1;
      for (let f = Math.ceil((pT + 0.6) * 12); f < 160; f++) {
        const u = Math.min(1, (f / 12 - (pT + 0.6)) / 1.1), v = Math.round(61 + 36 * (1 - Math.pow(1 - u, 2.2)));
        if (v === 97) { // 97% lands: the biggest hit
          k.play('impact', f / 12, { gain: 1.0, pan: 0.2 });
          k.play('ding', f / 12 + 0.01, { f: 'C6', gain: 0.42, pan: 0.2 });
          break;
        }
        if (v !== last) { k.play('tick', f / 12, { gain: 1.2, f: 2200 + (v - 61) * 20, pan: 0.25, minor: true }); last = v; }
      }
    }
    plot(pT + 1.75, pT + 2.05, 1.2, 0.25, false, 225); // dashed ring round the 97%

    // 5 TAKEAWAY
    letter('MORE EXAMPLES → BETTER GUESSES', kT, kT + 0.75, 0.85);
```

```
      k.play('success', kT + 0.75, { f: 'G5', gain: 0.7 }); // resolving landing
      plot(kT + 0.75, kT + 0.95, 1.15, 0, false, 300);
      // the head parks: one last quick run, then the motors settle
      k.tone(kT + 1.05, { f: [[0, 300], [0.25, 180]], type: 'square', dur: 0.3, gain: 0.025 * LV, a:
0.02, hold: 0.2, filter: 'lowpass', f2: 900, minor: true });
      k.play('tock', kT + 1.37, { f: 300, gain: 0.35, minor: true });
    },
  });
}
```